

Informe de Proyecto Final: Ingeniería en Sistemas de Información

Valentino Russo

8 de marzo de 2026

Resumen

En este documento se proveerá una descripción detallada de las tecnologías utilizadas para el desarrollo de KnowGraph y cómo fue desarrollado, un manual de usuario para conocer el comportamiento de las funcionalidades, y un manual para desarrolladores describiendo el funcionamiento del código.

Índice

1. Motivación y proyectos similares	4
1.1. Proyectos y herramientas relevantes	4
2. Repaso sobre grafos y cómo representar conocimiento con ellos	4
3. Linked Data	5
4. Grafos de Conocimiento	5
4.1. Definición y caracterización	5
4.2. Elementos esenciales	5
4.3. Modelos de datos y lenguajes	6
4.4. Creación, enriquecimiento y publicación	6
4.5. Aplicaciones de Grafos del Conocimiento	6
5. Knowledge Graph de Google	6
6. Resource Description Framework y su conexión con los grafos	7
6.1. Modelo de datos RDF	7
6.2. Formalización como Grafos	7
6.3. Propiedades semánticas y flexibilidad	8
6.4. Conexión con grafos de conocimiento	8
7. SPARQL	8
7.1. Principios fundamentales de SPARQL	8
7.2. Características principales	8
7.3. Aplicaciones de SPARQL	9
8. Apache Jena	10
8.1. Visión general y características principales	10
8.2. Arquitectura y modelos de datos	10
8.3. SPARQL y la API ARQ	11
8.4. Servidor SPARQL Fuseki	11
8.5. Almacenamiento con TDB y TDB2	11
8.6. Integración de Ontologías y razonamiento	11
8.7. Herramientas auxiliares y ecosistema	12
9. Maven y Apache jena	12
10.Spring para la creación de endpoints	12
11.Nginx	12

12.Docker	13
13.Vite, Typescript y React para construir un frontend profesional	13
13.1. Vite	14
13.2. TypeScript	14
13.3. React	14
14.Visualización de grafos	14
15.Visualización de Datos	14
15.1. Reducción de Dimensionalidad	15
15.2. Visualización en Navegadores Web	15
15.3. Implementación en Python	15
16.Comportamiento de la aplicación y manual de usuario	15
16.1. Ejecución de consultas	16
17.Sistema de Generación Visual de Consultas SPARQL	18
17.1. Introducción y Contexto	18
17.2. Arquitectura del Sistema	18
17.3. Flujo de Trabajo del Usuario	19
17.3.1. Paso 1: Configuración de Prefijos	19
17.3.2. Paso 2: Especificación de Conjuntos de Datos	19
17.3.3. Paso 3: Construcción de la Consulta	20
17.3.4. Paso 4: Configuración de Opciones Globales	21
17.4. Ejemplos de Uso	21
17.4.1. Ejemplo 1: Consulta Básica de Personas	21
17.4.2. Ejemplo 2: Consulta con Filtro y Agregación	21
17.4.3. Ejemplo 3: Consulta Federada con Grafo Nombrado	22
18.Constructor de triplas RDF para Consultas de Inserción	22
18.1. Introducción y Propósito	22
18.2. Arquitectura del Módulo	23
18.3. Flujo de Trabajo del Usuario	23
18.3.1. Paso 1: Configuración de Prefijos	23
18.3.2. Paso 2: Construcción de triplas Individuales	23
18.3.3. Paso 3: Gestión de Múltiples triplas	24
18.3.4. Paso 4: Selección del Grafo Destino	24
18.3.5. Paso 5: Generación de la Consulta	24
18.4. Ejemplos de Uso	25
18.4.1. Ejemplo 1: Inserción de Datos de Persona	25
18.4.2. Ejemplo 2: Inserción de Relaciones Organizacionales	25
18.4.3. Ejemplo 3: Inserción Masiva con Tipado	26
18.5. Características Avanzadas	26
18.6. Integración con el Sistema General	27
19.Visualización de Datos en KnowGraph	27
19.1. Visualización en forma de grafo	27
19.2. Visualización mediante reducción dimensional	29
20.Carga de datasets en KnowGraph	30
21.Importación de datos	31
22.Exportación de datos	33
23.Razonamiento sobre datos	34
23.1. Modelo de ontología	34
23.2. Modelo de inferencia	34
24.Documentación para desarrolladores	35
25.Introducción y Arquitectura General	35

26.Backend Java (Spring Boot y Apache Jena)	36
26.1. Apache Jena: Gestión del Grafo y Ejecución SPARQL	36
26.1.1. SparqlQueryResult y SparqlQueryResultData	36
26.1.2. Fábrica y Ejecutores de Consultas (Patrón Factory)	36
26.1.3. OntoManipulator e InferenceManipulator	37
26.1.4. SparqlService	37
26.2. Spring Boot: API REST y Configuración	37
26.2.1. ApplicationRunner	37
26.2.2. Controladores REST	37
26.3. Maven	38
27.Backend Python (Microservicios)	38
27.1. Importer (importer.py)	38
27.1.1. Propósito y Funcionalidad	38
27.1.2. Endpoints Principales	38
27.1.3. Flujo de Trabajo	38
27.2. Visualizer (visualizer.py)	38
27.2.1. Propósito y Funcionalidad	39
27.2.2. Algoritmos y Preprocesamiento	39
27.2.3. Flujo de la API	39
27.2.4. Estructura de Almacenamiento	39
28.Frontend (TypeScript, React y Vite)	39
28.1. Arquitectura General y Enrutamiento	40
28.1.1. Vistas Principales (Rutas)	40
28.2. Componentes de Visualización	41
28.2.1. Visualización de Grafos	41
28.2.2. Visualización de Reducción Dimensional	41
28.3. Componentes de Gestión de Datos y Consultas	41
28.3.1. Importación de Datos	41
28.3.2. Ejecución de Consultas	41
28.3.3. Construcción de Consultas	41
28.4. Componentes de Interfaz y Navegación	41
28.4.1. Barra de Navegación y Botones de Acción	41
29.Integración y Flujos de Trabajo	42
29.1. Comunicación entre Componentes	42
29.2. Flujo de Trabajo Típico	42
30.Docker: Contenerización y Orquestación de Servicios	42
30.1. Arquitectura de Contenedores	42
30.2. Configuración de Redes	43
30.3. Políticas de Reinicio	43
30.4. Flujo de Construcción y Despliegue	43
30.5. Ventajas de la Contenerización en este Proyecto	44
30.6. Consideraciones de Configuración	44
30.7. Ejemplo de Comunicación entre Contenedores	44
30.8. Configuración Individual de Contenedores	44
30.8.1. Frontend (React/Vite)	45
30.8.2. Backend Java (Spring Boot con Apache Jena)	45
30.8.3. Microservicios Python (Visualizer e Importer)	46
30.9. Integración de los Dockerfiles en el Flujo de Construcción	46
30.10 Consideraciones de Seguridad y Buenas Prácticas Implementadas	47
31.Uso de Nginx como proxy inverso	47
31.1. Arquitectura de red y servicios	47
31.2. Configuración del proxy inverso	47
31.3. Ventajas de este enfoque	48
31.4. Integración con Docker Compose	48
Bibliografía	49

1. Motivación y proyectos similares

KnowGraph es una aplicación diseñada para permitir la utilización de grafos de conocimiento de forma intuitiva y rápida, así como para realizar consultas sobre ellos y visualizar los resultados de manera clara y eficiente. Su finalidad es facilitar la exploración y explotación de datos interconectados sin requerir conocimientos profundos en lenguajes de consulta semántica o diseño de grafos de conocimiento.

Existen varios proyectos y herramientas relacionados con la gestión, consulta y visualización de conocimiento estructurado o grafos semánticos, cada uno con enfoques particulares que los hacen relevantes como antecedentes o comparativas para KnowGraph:

1.1. Proyectos y herramientas relevantes

- *Protégé*: es un editor de ontologías de código abierto y un sistema de gestión de conocimiento desarrollado por la Universidad de Stanford. Protégé permite definir modelos conceptuales formales (ontologías) y sus relaciones, facilitando la construcción de esquemas semánticos exportables a formatos como OWL y RDF, que pueden ser usados como base para grafos de conocimiento.
- *MindBugs Discovery*: es una plataforma que integra técnicas de inteligencia artificial, minería de datos y análisis semántico para ayudar a investigadores, periodistas y analistas a explorar y comprender narrativas complejas —particularmente aquellas relacionadas con desinformación— mediante visualizaciones interactivas y grafos que muestran conexiones entre entidades, eventos y datos verificados. La herramienta construye un conocimiento estructurado a partir de fuentes verificadas y permite explorar relaciones semánticas entre conceptos dentro de un grafo interactivo.
- *Stardog*: es una plataforma de base de datos semántica empresarial que permite almacenar y consultar grafos de conocimiento basados en RDF con capacidades avanzadas de razonamiento e integración de datos. Stardog incluye un motor que soporta consultas SPARQL complejas y razonamiento lógico sobre los datos, habilitando consultas federadas y análisis semántico profundo.
- *Wikidata Query Creator*: vinculado a *Wikidata*, una base de conocimiento abierta mantenida por la Fundación Wikimedia, permite realizar consultas complejas sobre una enorme red de entidades y relaciones utilizando SPARQL, facilitando la extracción de información interconectada y su análisis semántico.
- *DBpedia Query Creator*: asociado a *DBpedia*, un proyecto que extrae datos estructurados de Wikipedia y los convierte en un grafo de conocimiento accesible mediante consultas SPARQL. Esto permite explorar las relaciones semánticas entre miles de entidades extraídas de la enciclopedia, habilitando aplicaciones de consulta y visualización de datos.

Estas herramientas y plataformas representan enfoques diversos dentro del ecosistema de conocimiento estructurado y grafos de conocimiento: desde editores y modeladores de ontologías (*Protégé*), hasta plataformas que integran análisis semántico y visualizaciones avanzadas (*MindBugs Discovery*) o potentes motores de almacenamiento y consulta de conocimiento (*Stardog*, *Wikidata* y *DBpedia*). Todos ellos comparten el objetivo de organizar, interrogar y explotar datos interrelacionados, lo que los hace útiles para comparar, contrastar y fundamentar el desarrollo de KnowGraph.

2. Repaso sobre grafos y cómo representar conocimiento con ellos

Un grafo es una estructura matemática utilizada para modelar relaciones entre entidades. Formalmente, un grafo se compone de un conjunto de nodos (o vértices), que representan entidades u objetos, y un conjunto de aristas, que representan las relaciones existentes entre dichos nodos. Dependiendo de la naturaleza de las relaciones, los grafos pueden ser dirigidos o no dirigidos, así como etiquetados o no etiquetados.

En el ámbito de la representación del conocimiento, los grafos resultan especialmente adecuados debido a su capacidad para modelar de manera explícita las relaciones semánticas entre conceptos. Cada nodo representa a una entidad, un concepto o un recurso, mientras que cada arista indica

el tipo de relación que los conecta, permitiendo así una estructura flexible y extensible para la organización de la información.

3. Linked Data

Linked Data es un conjunto de buenas prácticas para la publicación y conexión de datos estructurados en la Web, con el objetivo de facilitar su reutilización e interoperabilidad. Este enfoque se basa en el uso de estándares abiertos que permiten que los datos sean comprensibles tanto para humanos como para máquinas, promoviendo una Web de datos interconectados en lugar de documentos aislados.

Los principios fundamentales de Linked Data incluyen el uso de identificadores URI para nombrar recursos, el acceso a dichos recursos mediante el protocolo HTTP, la representación de la información utilizando modelos de datos estándar como RDF, y la inclusión de enlaces hacia otros recursos relacionados. Estos principios permiten integrar información proveniente de diferentes fuentes, creando un ecosistema de datos enlazados que puede ser explorado y combinado de manera automática.

La adopción de Linked Data aporta beneficios como una mayor interoperabilidad entre sistemas, la mejora en la calidad y el contexto de los datos, y la posibilidad de descubrir nuevo conocimiento a partir de la conexión entre conjuntos de datos distribuidos. Por estas razones, Linked Data es un componente clave dentro de la Web Semántica y de las iniciativas modernas de gestión y publicación de datos.

4. Grafos de Conocimiento

Los *grafos de conocimiento* son estructuras de representación del conocimiento que combinan la flexibilidad de los modelos de grafos con la semántica explícita de las relaciones y entidades, permitiendo modelar, integrar y explotar grandes volúmenes de información heterogénea. Han experimentado un crecimiento notable tanto en la investigación académica como en aplicaciones industriales, debido a su capacidad para representar datos interconectados a gran escala y facilitar operaciones de razonamiento, integración y consulta sobre estos datos.

4.1. Definición y caracterización

Aunque no existe una definición única universalmente aceptada, los grafos de conocimiento se pueden entender, en términos generales, como estructuras de datos que representan *entidades* explícitas y las *relaciones* entre ellas mediante nodos y aristas en un grafo semántico. Además, incorporan mecanismos formales de vocabulario (esquemas o ontologías) y, en muchos casos, capacidades de razonamiento automático para derivar conocimiento implícito a partir de datos explícitos.

En la literatura técnica revisada por Hogan et al., se identifican diversas perspectivas sobre qué constituye un grafo de conocimiento. Algunas definiciones se centran en componentes técnicos específicos, como una combinación de:

- un componente extensional de datos y esquemas modelados como grafos,
- un componente intensional de reglas o axiomas para inferencia,
- un componente derivado generado mediante procesos de razonamiento sobre los anteriores

Este enfoque metodológico integra tanto la representación explícita de hechos (triplas) como la capacidad para inferir nuevos hechos conforme a reglas lógicas predefinidas, lo que extiende significativamente el alcance de los grafos de conocimiento más allá de simples redes de relaciones.

4.2. Elementos esenciales

Los grafos de conocimiento típicamente incluyen los siguientes elementos:

- *Entidades*: Representan objetos del dominio de interés (personas, lugares, conceptos, eventos, etc.).
- *Relaciones*: Aristas etiquetadas que describen interacciones o conexiones significativas entre entidades.

- *Atributos y literales*: Valores asociados a entidades o relaciones que describen propiedades concretas.
- *Esquemas u ontologías*: Vocabularios formales que definen clases, propiedades y restricciones, proporcionando una base semántica para la interpretación de los datos.

Estas estructuras suelen permitir tanto la consulta explícita de información como operaciones de razonamiento que derivan nuevo conocimiento a partir de hechos existentes, contribuyendo a la construcción de sistemas de inteligencia más ricos y expresivos.

4.3. Modelos de datos y lenguajes

Los grafos de conocimiento pueden formalizarse mediante modelos de datos basados en grafos dirigidos etiquetados, donde cada arista representa una relación semántica entre dos nodos etiquetados. Para la consulta y manipulación de estos datos, se emplean lenguajes de consulta específicos como SPARQL (discutido en la Sección anterior), que permiten describir patrones de grafos y recuperar información de manera declarativa sobre grafos RDF integrados en los grafos de conocimiento.

4.4. Creación, enriquecimiento y publicación

La construcción de un grafo de conocimiento implica varias fases interrelacionadas:

- *Extracción de conocimiento*: Identificación de entidades y relaciones relevantes a partir de fuentes heterogéneas de datos.
- *Enriquecimiento y alineamiento*: Integración de datos nuevos y alineamiento con vocabularios o esquemas existentes.
- *Evaluación de la calidad*: Verificación de consistencia, completitud y corrección de las declaraciones en el grafo.
- *Publicación y mantenimiento*: Difusión del grafo de conocimiento a través de servicios, APIs o endpoints de consulta y su actualización continua conforme evolucionan los datos subyacentes.

Estas fases son críticas para garantizar que los grafos de conocimiento sean útiles, precisos y sostenibles en aplicaciones reales, especialmente cuando se trabaja con datos dinámicos o de gran escala.

4.5. Aplicaciones de Grafos del Conocimiento

Los grafos de conocimiento han sido adoptados en numerosos dominios, incluyendo:

- motores de búsqueda y asistentes inteligentes, que integran conocimiento semántico para mejorar la relevancia de las respuestas,
- sistemas de recomendación y personalización basados en relaciones semánticas profundas,
- integración de datos en empresas que gestionan múltiples fuentes de información heterogéneas,
- soporte para razonamiento automático y explicabilidad en sistemas de inteligencia artificial.

5. Knowledge Graph de Google

El Knowledge Graph de Google es una base de conocimiento semántico introducida en 2012 con el objetivo de mejorar la comprensión de las consultas realizadas por los usuarios y ofrecer resultados de búsqueda más relevantes y contextuales. A diferencia de los motores de búsqueda tradicionales, centrados principalmente en la coincidencia de palabras clave, el Knowledge Graph se basa en la identificación de entidades del mundo real —como personas, lugares, organizaciones y conceptos— y en el modelado de las relaciones que existen entre ellas.

Este sistema permite a Google interpretar la intención detrás de una consulta y establecer conexiones semánticas entre distintos elementos de información, integrando datos provenientes de múltiples fuentes estructuradas y no estructuradas. De este modo, el Knowledge Graph posibilita

la generación de resultados enriquecidos, como paneles de información, respuestas directas y sugerencias relacionadas, que proporcionan al usuario una visión más completa y contextualizada del tema buscado.

Desde el punto de vista conceptual, el Knowledge Graph adopta principios fundamentales de la Web Semántica, como el uso de identificadores únicos para representar entidades y la definición explícita de relaciones entre ellas. Sin embargo, a diferencia de las iniciativas clásicas de Linked Data, el Knowledge Graph no se presenta como un conjunto de datos abiertos ni completamente accesibles, sino como una infraestructura propietaria controlada por Google.

La influencia del Knowledge Graph se extiende también al ámbito del desarrollo web y la optimización para motores de búsqueda (SEO). A través de iniciativas como *schema.org*, Google incentiva a los propietarios de sitios web a describir su contenido mediante datos estructurados, facilitando así la extracción de significado y la asociación de dicho contenido con las entidades presentes en su base de conocimiento. Este enfoque ha contribuido a la adopción masiva de prácticas inspiradas en la Web Semántica, aunque bajo un modelo centralizado.

6. Resource Description Framework y su conexión con los grafos

El *Resource Description Framework (RDF)* es un estándar recomendado por el *World Wide Web Consortium (W3C)* para la *representación y el intercambio de información en la Web Semántica*. Fue desarrollado con el propósito de proporcionar un *modelo de datos flexible, extensible y orientado a grafos* que permita describir recursos y sus relaciones de forma que puedan ser interpretados y procesados por máquinas de manera interoperable en entornos distribuidos. El estándar RDF ha evolucionado a lo largo del tiempo, con versiones aprobadas como RDF 1.0 y RDF 1.1, consolidándose como una de las piedras angulares de la Web Semántica y la infraestructura de datos enlazados (*Linked Data*).

6.1. Modelo de datos RDF

La base del modelo RDF es un *grafo dirigido etiquetado* compuesto por estructuras denominadas *triplas* o *triples* (del inglés *triples*): cada tripla está formada por un *sujeto*, un *predicado* y un *objeto*, que conjuntamente representan una *afirmación semántica* sobre algún dominio de conocimiento. En esta representación:

- El *sujeto* identifica el recurso que se está describiendo.
- El *predicado* (o propiedad) indica la *relación* o característica del sujeto.
- El *objeto* corresponde al valor de esa característica o al recurso con el que se relaciona el sujeto.

Cada uno de estos componentes puede ser identificado mediante un *Identificador de Recursos Uniforme (URI)*, lo que permite referenciar inequívocamente recursos distribuidos a lo largo de la Web. El objeto también puede ser un *valor literal* (por ejemplo, una cadena de texto, un número o una fecha).

Conceptualmente, un conjunto de triples puede representarse como un *grafo RDF* en el que los nodos representan recursos o literales y las aristas etiquetadas representan las relaciones semánticas entre ellos. Esta estructura de grafo posibilita la representación de información compleja y altamente interconectada de forma natural y escalable.

6.2. Formalización como Grafos

El modelo de datos RDF se define formalmente en términos de grafos abstractos, en los cuales cada tripla constituye una *arista dirigida y etiquetada* que conecta dos nodos. El conjunto de todas las triplas constituye un *grafo RDF*, que puede visualizarse como una red de relaciones entre entidades. Esta abstracción matemática permite aplicar técnicas de teoría de grafos y lógica para razonamiento, integración y consulta de información.

Un *grafo RDF* puede describir hechos simples, como por ejemplo:

(“Juan”, “tieneNombre”, “Juan Pérez”)

Esto se traduce en un nodo para “Juan”, un nodo literal para “Juan Pérez”, y una arista etiquetada con la propiedad “tieneNombre” que los enlaza.

6.3. Propiedades semánticas y flexibilidad

Una de las ventajas fundamentales de RDF es su *capacidad para representar conocimiento sin imponer un esquema rígido* ni una ontología prescrita. Los grafos RDF pueden crecer de forma incremental y pueden integrar datos de múltiples fuentes heterogéneas, lo que facilita la *interoperabilidad*, la *extensibilidad* y la *reutilización* de información. Asimismo, dado que los nombres de recursos y propiedades están basados en URIs, es posible establecer enlaces entre datos distribuidos, característica esencial del paradigma *Linked Data*.

El modelo RDF también permite describir los datos en diferentes *formatos de serialización*, como RDF/XML, Turtle, N-Triples, JSON-LD, entre otros. Estos formatos permiten tanto la representación textual como la transmisión de datos RDF de manera legible por máquinas y estandarizada.

6.4. Conexión con grafos de conocimiento

La representación de información como grafos RDF es un *elemento central para la construcción de grafos de conocimiento*, ya que permite modelar de forma explícita las entidades y sus relaciones semánticas. Un *grafo de conocimiento* no es más que un conjunto de recursos interconectados mediante relaciones, formalizado como un *grafo RDF enriquecido con vocabularios y ontologías* que aportan semántica adicional. Esta estructura facilita no solo la *recuperación eficiente de información* mediante lenguajes de consulta como SPARQL, sino también la *inferencias de nuevo conocimiento*, especialmente cuando se combinan RDF con esquemas de ontologías (por ejemplo, RDFS y OWL) y mecanismos de razonamiento semántico.

En este sentido, RDF actúa como una *capa base de representación* que proporciona la semántica estructural necesaria para que sistemas automatizados puedan integrar, enlazar y explotar datos de manera coherente en la Web Semántica. Esta capacidad de formalizar el significado y las relaciones entre recursos convierte a RDF en un pilar fundamental para la Web de Datos, facilitando aplicaciones avanzadas como motores de búsqueda semántica, asistentes inteligentes, análisis de redes de información y sistemas de integración de datos a gran escala.

7. SPARQL

SPARQL (SPARQL Protocol and RDF Query Language) es el lenguaje de consulta y actualización estándar definido por el World Wide Web Consortium (W3C) para datos representados mediante el modelo RDF (Resource Description Framework). Su diseño responde a la necesidad de consultar información estructurada como grafos semánticos, permitiendo expresar relaciones complejas entre recursos identificados por URIs. En el contexto de la Web Semántica y los grafos de conocimiento, SPARQL constituye el principal mecanismo de acceso, exploración y transformación de datos.

De acuerdo con DuCharme (*Learning SPARQL, 2nd Edition*), SPARQL debe entenderse no solo como un lenguaje de recuperación de datos, sino como una herramienta declarativa para el análisis de grafos, la integración de fuentes distribuidas y la construcción de nuevas representaciones semánticas a partir de datos existentes.

7.1. Principios fundamentales de SPARQL

SPARQL se basa en la coincidencia de patrones de grafos contra un conjunto de triplas RDF. Cada consulta describe un patrón que el motor SPARQL intenta emparejar con el grafo de datos, produciendo un conjunto de soluciones que satisfacen las restricciones declaradas.

A diferencia de los modelos relacionales, donde las consultas se expresan sobre tablas, SPARQL opera directamente sobre la estructura del grafo, lo que permite navegar relaciones explícitas e implícitas sin necesidad de esquemas rígidos. Este enfoque favorece la flexibilidad, la extensibilidad y la interoperabilidad entre conjuntos de datos heterogéneos.

7.2. Características principales

- *Lenguaje declarativo orientado a grafos*: SPARQL permite describir qué información se desea obtener sin especificar cómo debe ejecutarse la consulta. Los patrones de triplas representan fragmentos del grafo RDF que deben coincidir con los datos almacenados.

- *Sintaxis inspirada en SQL*: Aunque conceptualmente distinto, SPARQL adopta una sintaxis familiar mediante cláusulas como **SELECT**, **WHERE**, **FILTER**, **OPTIONAL** y **UNION**, facilitando su comprensión por parte de usuarios con experiencia en bases de datos relacionales.
- *Patrones de triplas y variables*: El núcleo de una consulta SPARQL está formado por patrones de la forma:

[H]
 ?sujeto ?predicado ?objeto .

donde las variables permiten capturar valores RDF que cumplen con las restricciones del patrón.

- *Formas de consulta*:
 - **SELECT**: Recupera un conjunto de resultados en forma tabular, vinculando variables a valores RDF.
 - **CONSTRUCT**: Produce un nuevo grafo RDF a partir de las soluciones obtenidas, permitiendo la transformación y el enriquecimiento de datos.
 - **ASK**: Evalúa la existencia de al menos una solución que satisfaga el patrón de consulta.
 - **DESCRIBE**: Genera un grafo RDF que describe recursos específicos, dependiendo de la implementación del motor SPARQL.
- *Extensiones de SPARQL 1.1*: La especificación SPARQL 1.1 amplía significativamente las capacidades del lenguaje mediante:
 - Expresiones y funciones integradas (**BIND**, **IF**, **COALESCE**, **STR**, **LANG**, **DATATYPE**).
 - Operaciones de agregación y agrupamiento (**COUNT**, **SUM**, **AVG**, **GROUP BY**, **HAVING**).
 - Subconsultas y consultas anidadas.
 - Operadores de exclusión y negación (**MINUS**, **NOT EXISTS**).
 - Mecanismos de ordenamiento y paginación (**ORDER BY**, **LIMIT**, **OFFSET**).
 - Consultas federadas mediante **SERVICE**, que permiten acceder a endpoints SPARQL distribuidos.

7.3. Aplicaciones de SPARQL

- *Consulta de repositorios RDF y triplestores*: SPARQL es el lenguaje estándar para interactuar con sistemas de almacenamiento RDF como Apache Jena, Virtuoso, GraphDB, Blazegraph y Stardog.
- *Integración semántica de datos*: Gracias al uso de URIs y vocabularios compartidos, SPARQL facilita la integración de datos heterogéneos provenientes de múltiples dominios y fuentes, favoreciendo la interoperabilidad semántica.
- *Exploración y análisis de grafos de conocimiento*: SPARQL se utiliza extensamente en ontologías basadas en RDFS y OWL, así como en iniciativas de datos abiertos y grafos de conocimiento de gran escala, como DBpedia y Wikidata.
- *Transformación y generación de conocimiento*: Mediante consultas **CONSTRUCT**, SPARQL permite derivar nuevos grafos RDF, extraer subgrafos relevantes y preparar datos para procesos posteriores de análisis o visualización.
- *Soporte para inferencia y razonamiento semántico*: Combinado con motores de inferencia, SPARQL permite formular consultas que explotan conocimiento implícito, como jerarquías de clases y propiedades, fortaleciendo las capacidades analíticas de los sistemas semánticos.

8. Apache Jena

Apache Jena es un *marco de trabajo de código abierto para Java* diseñado para construir aplicaciones orientadas a la Web Semántica y Linked Data. Proporciona *APIs completas para la manipulación, almacenamiento, consulta y publicación de datos RDF*, así como soporte para ontologías, razonamiento y servicios SPARQL. Su diseño modular permite integrar múltiples componentes especializados que, en conjunto, constituyen una plataforma potente para desarrollar soluciones basadas en grafos de conocimiento y tecnologías semánticas.

8.1. Visión general y características principales

Apache Jena ofrece una arquitectura compuesta por varios subsistemas interrelacionados que permiten cubrir todo el ciclo de vida de los datos RDF y su explotación:

- *RDF API*: Es la API central de Jena para crear, manipular y serializar grafos RDF. Proporciona abstracciones de más alto nivel como `Resource`, `Literal`, `Statement` y `Model`, facilitando la construcción y consulta de triplas RDF en memoria o en almacenamiento persistente.
- *SPARQL (ARQ)*: Jena integra un motor de consulta y actualización compatible con SPARQL 1.1 llamado ARQ, lo que permite ejecutar consultas SPARQL sobre modelos RDF locales o remotos y sobre datasets persistentes.
- *TDB/TDB2*: Son componentes de almacenamiento de triplas RDF de alto rendimiento diseñados para persistir datos directamente en disco. Jena incluye TDB2 como su versión más reciente y optimizada de almacenamiento persistente, soportando transacciones y manejo eficiente de grandes volúmenes de datos RDF.
- *Fuseki*: Es un *servidor SPARQL* que expone datasets RDF mediante *endpoints HTTP*, permitiendo consultas SPARQL y actualizaciones remotas conforme a los estándares SPARQL 1.1 y SPARQL Graph Store Protocol. El servidor puede ejecutarse como servicio independiente, aplicación web empaquetada (WAR) o integrado en otras aplicaciones.
- *Ontología e inferencia*: Jena provee APIs específicas para trabajar con *ontologías OWL y RDFS*, así como un motor de inferencia que permite derivar conocimiento implícito a partir de hechos explícitos, mediante reglas de razonamiento predefinidas o personalizadas.
- *Otros módulos auxiliares*: Jena incluye funcionalidades adicionales como procesadores SHACL y ShEx para validación de esquemas, soporte GeoSPARQL para datos geoespaciales, búsqueda de texto indexado (Lucene/Solr), APIs para conexiones unificadas a datasets (`RDFConnection`), herramientas de línea de comandos y generadores de código (`QueryBuilder`).

8.2. Arquitectura y modelos de datos

La esencia de Jena radica en representar datos RDF como *grafos dirigidos de triplas* compuestos por sujetos, predicados y objetos. Su API de RDF abstrae estas estructuras de forma que aplicaciones Java puedan:

- crear y modificar grafos RDF,
- buscar patrones de triplas,
- cargar y serializar datos en formatos estándar como Turtle, RDF/XML o N-Triples,
- integrar múltiples modelos en un dataset cohesivo.

El subsistema de inferencia permite *enriquecer datos* utilizando sistemas de reglas, ya sean basados en RDFS/OWL o personalizados, de modo que las conclusiones derivadas se reflejen en los modelos de forma transparente para el desarrollador.

8.3. SPARQL y la API ARQ

Jena incluye el motor ARQ para manejar consultas SPARQL conforme a las especificaciones actuales del W3C. ARQ soporta:

- consultas SELECT, CONSTRUCT, ASK y DESCRIBE,
- operaciones de actualización SPARQL 1.1,
- consultas federadas y operaciones de graph store,
- integración con modelos locales o con endpoints remotos.

Además, la API `RDFConnection` unifica el acceso a datasets RDF, ya sea localmente o a través de servidores SPARQL como Fuseki, facilitando operaciones de consulta, actualización y gestión de datos en una única interfaz.

8.4. Servidor SPARQL Fuseki

Fuseki es el componente de Jena diseñado para publicar datos RDF como servicios web semánticos. Entre sus características destacan:

- implementación de los protocolos SPARQL 1.1 Query y Update, y SPARQL Graph Store Protocol, lo que permite ejecutar consultas y modificaciones sobre HTTP de forma estándar.
- integración con TDB para almacenamiento persistente y transaccional de datasets, ofreciendo robustez y escalabilidad.
- disponibilidad de una interfaz de usuario para monitorización, administración y ejecución de consultas de forma interactiva.
- despliegue flexible como servidor independiente, aplicación web o dentro de infraestructura corporativa.

Fuseki se posiciona como una pieza clave para *servir grafos de conocimiento en entornos distribuidos*, habilitando a los consumidores de datos (otros sistemas o aplicaciones) a acceder y manipular datos RDF de forma remota sin acoplamiento directo a los formatos de almacenamiento subyacentes.

8.5. Almacenamiento con TDB y TDB2

TDB y su versión evolucionada TDB2 son módulos de almacenamiento persistente en Jena que:

- ofrecen un triplestore de alto rendimiento con soporte transaccional, asegurando propiedades ACID en operaciones de lectura y escritura.
- permiten manejar datasets RDF de grandes dimensiones directamente en disco, aprovechando mecanismos de indexación eficientes.
- se integran con Fuseki para proporcionar acceso remoto a estos datasets sin comprometer la integridad o la coherencia de los datos.

8.6. Integración de Ontologías y razonamiento

Además de la API base para RDF, Jena proporciona soporte para ontologías definidas en RDFS y OWL. Esto incluye:

- abstracciones de clases, propiedades y axiomas para modelar conceptos complejos dentro de un dominio,
- motores de inferencia que permiten derivar relaciones y clases implícitas conforme a las especificaciones semánticas de estas lenguas,
- conectores que permiten enlazar el razonamiento interno con sistemas externos especializados si se requiere mayor potencia de inferencia.

8.7. Herramientas auxiliares y ecosistema

Jena incluye numerosas herramientas y extensiones que facilitan la manipulación de datos RDF, entre ellas:

- módulos para validación de esquemas (SHACL, ShEx),
- soporte para búsqueda de texto con indexación externa (Lucene, Solr),
- utilidades de línea de comandos para administración y pruebas,
- documentación generada automáticamente (JavaDoc) que cubre todos los APIs y subsistemas.

9. Maven y Apache jena

Apache Maven es una herramienta de gestión y automatización de proyectos en Java que facilita la administración de dependencias, la compilación y el empaquetado de aplicaciones. Cuando se desarrolla con Apache Jena en un proyecto Java, Maven permite declarar las bibliotecas necesarias directamente en el archivo `pom.xml`, evitando la inclusión manual de múltiples archivos JAR y asegurando que todas las dependencias transitivas de Jena se resuelvan automáticamente.

Para utilizar Jena en un proyecto Maven, basta con añadir las dependencias correspondientes a la sección `<dependencies>` del `pom.xml`. Por ejemplo, la inclusión del artefacto `apache-jena-libs` permite importar de forma centralizada las bibliotecas principales de Jena (como `jena-core`, `jena-arq`, `jena-tdb`, etc.) sin tener que gestionarlo individualmente.

10. Spring para la creación de endpoints

Spring Boot es un framework del ecosistema Java que se utilizó para el desarrollo de la capa backend de la aplicación. Su principal ventaja radica en la simplificación de la configuración y puesta en marcha de aplicaciones basadas en Spring, permitiendo centrarse en la lógica de negocio sin necesidad de una configuración extensa.

En el contexto de este trabajo, Spring Boot fue empleado para la creación de endpoints REST, los cuales actúan como punto de comunicación entre el frontend y el backend del sistema. Estos endpoints permiten recibir solicitudes HTTP, procesar los datos correspondientes y devolver respuestas estructuradas, generalmente en formato JSON.

El framework facilita la definición de estos endpoints mediante el uso de anotaciones como `@RestController`, `@RequestMapping`, `@GetMapping` y `@PostMapping`, lo que mejora la legibilidad del código y reduce la complejidad del desarrollo. Además, Spring Boot incorpora de forma nativa un servidor embebido, lo que elimina la necesidad de configurar un servidor externo para la ejecución de la aplicación.

Asimismo, Spring Boot proporciona mecanismos integrados para el manejo de dependencias, la validación de datos y el tratamiento de excepciones, contribuyendo a la creación de una arquitectura robusta, mantenible y escalable. Estas características hacen que Spring Boot sea una elección adecuada para el desarrollo de servicios web modernos y orientados a APIs REST.

11. Nginx

Nginx es un servidor web de alto rendimiento y código abierto que también funciona como proxy inverso, balanceador de carga, caché de contenido y proxy para protocolos TCP/UDP y correo. Fue creado por Igor Sysoev y se distribuye bajo la licencia BSD de 2 cláusulas. Está diseñado para manejar un gran número de conexiones concurrentes con un consumo mínimo de recursos, lo que lo convierte en una opción popular para sitios web y aplicaciones de alta demanda.

Como proxy inverso, Nginx se sitúa entre los clientes y los servidores backend, interceptando las solicitudes y decidiendo cómo manejarlas. Esta arquitectura ofrece múltiples ventajas:

- **Balanceo de carga:** Distribuye el tráfico entrante entre varios servidores backend utilizando algoritmos como round-robin, least connections o IP hash, evitando la sobrecarga de un único servidor y mejorando la disponibilidad.

- **Caché de contenido:** Almacena respuestas de los servidores backend (como páginas estáticas o resultados de consultas) y las sirve directamente a los clientes, reduciendo la carga en los servidores y acelerando los tiempos de respuesta.
- **Terminación SSL/TLS:** Gestiona el cifrado y descifrado de las conexiones seguras, liberando a los servidores backend de esta tarea y centralizando la gestión de certificados .
- **Seguridad:** Oculta la infraestructura interna al actuar como intermediario, protegiendo los servidores backend de ataques directos y permitiendo implementar políticas de seguridad centralizadas, como listas de denegación o limitación de tasa.

Nginx utiliza un modelo de procesos asíncrono y basado en eventos, con un proceso maestro que gestiona múltiples procesos worker, los cuales realizan el procesamiento real de las solicitudes . Su configuración es flexible y modular, organizada en directivas y contextos (como `http`, `server`, `location`) que permiten un control preciso del comportamiento del servidor.

12. Docker

Docker es una plataforma de código abierto diseñada para automatizar el despliegue de aplicaciones dentro de contenedores de software . Los contenedores son entornos ligeros y portátiles que empaquetan una aplicación junto con todas sus dependencias, bibliotecas y archivos de configuración, garantizando que se ejecute de manera uniforme en cualquier sistema que tenga Docker instalado, independientemente de las diferencias en la infraestructura subyacente .

A diferencia de las máquinas virtuales tradicionales, los contenedores comparten el núcleo del sistema operativo anfitrión, lo que los hace mucho más eficientes en términos de recursos y tiempo de inicio . Docker proporciona herramientas para construir imágenes de contenedor, distribuir las a través de registros (como Docker Hub) y gestionar su ciclo de vida mediante comandos sencillos .

En el contexto de este proyecto, Docker se utiliza para contenerizar cada uno de los servicios que componen la aplicación: el frontend React, el backend Java con Apache Jena, y los microservicios Python de importación y visualización. Esta aproximación ofrece múltiples ventajas:

- **Aislamiento:** Cada servicio se ejecuta en su propio contenedor, con sus propias dependencias, evitando conflictos entre versiones de bibliotecas o entornos de ejecución.
- **Portabilidad:** Las imágenes de Docker pueden ejecutarse en cualquier máquina que tenga Docker, desde el ordenador de un desarrollador hasta servidores en la nube, facilitando el despliegue y la colaboración.
- **Escalabilidad:** Los servicios pueden escalarse de forma independiente añadiendo más instancias de un contenedor, especialmente útil para los microservicios de visualización que pueden requerir mayor capacidad de procesamiento.
- **Consistencia:** El entorno de desarrollo, prueba y producción es idéntico, eliminando los problemas de “funciona en mi máquina”.
- **Gestión simplificada:** Con Docker Compose se define y orquesta toda la arquitectura de servicios en un único archivo YAML, permitiendo levantar el sistema completo con un solo comando.

La adopción de Docker ha sido fundamental para lograr una arquitectura de microservicios desacoplada, fácil de mantener y desplegar.

13. Vite, Typescript y React para construir un frontend profesional

En el desarrollo frontend moderno, Vite, TypeScript y React son tecnologías que, utilizadas en conjunto, permiten crear aplicaciones web profesionales, escalables y de alto rendimiento. A continuación se presenta una breve explicación de cada una de estas tecnologías y su rol en el desarrollo de interfaces de usuario.

13.1. Vite

Vite es una herramienta de construcción (*build tool*) y servidor de desarrollo moderna orientada a proyectos web. Fue creada para proporcionar un flujo de desarrollo rápido y eficiente aprovechando los módulos ES nativos del navegador. Durante el desarrollo, Vite actúa como un servidor que sirve los archivos directamente sin necesidad de realizar un empaquetado completo, permitiendo tiempos de arranque casi instantáneos y recargas en caliente (Hot Module Replacement, HMR). Esto mejora considerablemente la experiencia del desarrollador. Para producción, Vite utiliza herramientas como Rollup para generar paquetes optimizados, con código más pequeño y eficiente.

13.2. TypeScript

TypeScript es un lenguaje de programación de alto nivel que se basa en JavaScript pero añade tipado estático opcional. Fue desarrollado por Microsoft y permite a los desarrolladores definir tipos de datos para variables, parámetros y estructuras más complejas. Este sistema de tipos se verifica en tiempo de compilación, lo que ayuda a detectar errores antes de ejecutar la aplicación y mejora la mantenibilidad y claridad del código. TypeScript se transpila a JavaScript estándar para ser ejecutado por los navegadores o entornos como Node.js.

13.3. React

React es una biblioteca de JavaScript de código abierto diseñada para construir interfaces de usuario (*user interfaces*). Fue desarrollada por Meta (antes Facebook) y se utiliza ampliamente para crear aplicaciones web interactivas y dinámicas. React se basa en una arquitectura de componentes reutilizables: cada componente representa una parte específica de la interfaz de usuario y puede gestionar su propio estado. Además, React utiliza una representación virtual del DOM (*Virtual DOM*) para actualizar de forma eficiente solo las partes de la interfaz que han cambiado, reduciendo el trabajo innecesario al interactuar con la página.

La combinación de estas tres tecnologías conforma una base sólida para construir aplicaciones frontend profesionales, ya que aprovecha la velocidad de desarrollo, la seguridad de tipos y la modularidad de componentes.

14. Visualización de grafos

Para la visualización de grafos tanto en dos dimensiones (2D) como en tres dimensiones (3D) se empleó la librería *ForceGraph*. Esta herramienta permite definir un grafo a partir de una lista de nodos y aristas, a partir de la cual se genera una simulación física que posibilita la disposición automática de los elementos y su posterior renderizado completo.

El motor de simulación física subyacente utilizado por *ForceGraph* es *d3-force*, el cual implementa algoritmos de fuerzas para determinar la posición de los nodos en el espacio. En el caso de la visualización tridimensional, el renderizado se realiza mediante la librería *Three.js*, apoyándose en la tecnología *WebGL* para el aprovechamiento de la aceleración gráfica por hardware. Por otro lado, la visualización bidimensional se lleva a cabo utilizando tecnologías estándar de *HTML5*.

Ambas implementaciones de *ForceGraph*, tanto en 2D como en 3D, ofrecen un amplio conjunto de opciones de configuración que permiten personalizar el comportamiento y la apariencia de las visualizaciones. Estas opciones incluyen, entre otras, el control de la cámara, la definición de colores y tamaños de nodos y aristas, así como la parametrización de las fuerzas que rigen la simulación, lo que permite adaptar la visualización a distintos tipos de grafos y necesidades de análisis.

15. Visualización de Datos

La visualización de datos multivariados, es decir, datos con tres o más variables, resulta esencial para explorar patrones, relaciones y estructuras presentes en conjuntos de datos complejos. Sin embargo, la representación directa de datos de alta dimensión (mayores que tres) no es posible de forma intuitiva para el ser humano. Por ello, es necesario aplicar técnicas de reducción de dimensionalidad que transformen estos datos originales a espacios de menor dimensión —generalmente bidimensionales (2D) o tridimensionales (3D)— sin perder las características significativas del conjunto de datos original. Estas técnicas permiten condensar la información para facilitar su interpretación visual.

15.1. Reducción de Dimensionalidad

La reducción de dimensionalidad consiste en proyectar un conjunto de datos de alta dimensión en un espacio de menor dimensión preservando, en la medida de lo posible, las relaciones relevantes entre los puntos de datos. Entre los métodos más empleados en visualización se encuentran:

- *Análisis de Componentes Principales (PCA)*: Técnica lineal que busca nuevas direcciones ortogonales (componentes principales) que expliquen la máxima varianza de los datos. La proyección en estas componentes permite reducir la dimensión original mientras se conserva la mayor parte de la información cuantitativa de los datos .
- *t-Distributed Stochastic Neighbor Embedding (t-SNE)*: Algoritmo no lineal diseñado específicamente para visualización de datos en espacios de baja dimensión. Su objetivo es preservar las relaciones locales entre puntos cercanos en el espacio de alta dimensión, asignando a cada punto una ubicación en un espacio 2D o 3D de forma que los puntos similares queden próximos entre sí .
- *Uniform Manifold Approximation and Projection (UMAP)*: Técnica basada en teoría de variedades y topología, capaz de capturar tanto la estructura local como global de los datos. UMAP ofrece una reducción de dimensionalidad eficiente y escalable, y suele generar proyecciones que preservan la estructura de los datos de forma más consistente incluso en grandes conjuntos de datos .

Estas técnicas permiten obtener un conjunto de puntos en 2D o 3D que puede ser utilizado para elaborar visualizaciones significativas y detectar patrones, grupos o asociaciones dentro de los datos .

15.2. Visualización en Navegadores Web

Una vez realizada la reducción de dimensionalidad, los resultados deben ser representados mediante tecnologías que permitan visualizar y manipular los puntos de datos de forma interactiva en navegadores web. Para ello, se emplean bibliotecas especializadas en gráficos:

- *D3.js (Data-Driven Documents)*: Biblioteca de JavaScript para crear visualizaciones dinámicas en 2D, utilizando estándares web como SVG, HTML y CSS. D3.js facilita la vinculación directa entre los datos y los elementos visuales, permitiendo la generación de diagramas de dispersión, ejes, colores y otras representaciones visuales basadas en datos.
- *Three.js*: Biblioteca de JavaScript basada en WebGL que permite renderizar gráficos tridimensionales en tiempo real. Three.js es especialmente útil para visualizar conjuntos de puntos en 3D, ofreciendo soporte para rotación, zoom, iluminación y animaciones, lo que potencia la exploración interactiva de los datos reducidos.

El uso de D3.js y Three.js permite transformar los puntos reducidos en gráficos interactivos que pueden ser explorados mediante un navegador web, facilitando la interpretación visual de patrones complejos en datos de múltiples variables.

15.3. Implementación en Python

La aplicación de técnicas de reducción de dimensionalidad en Python se realiza comúnmente mediante librerías especializadas como `scikit-learn` o `umap-learn`, que proveen implementaciones de PCA, t-SNE y UMAP. Estas herramientas permiten ajustar los datos originales y transformar las coordenadas al espacio reducido deseado, generando así el conjunto de puntos que será visualizado con D3.js o Three.js.

16. Comportamiento de la aplicación y manual de usuario

Una vez desarrollados los conceptos teóricos necesarios, a continuación se describe el funcionamiento de la aplicación *KnowGraph* que va a servir como manual de usuario de la aplicación.

16.1. Ejecución de consultas

Al acceder a la aplicación mediante `http://localhost/`, la primera interfaz que se presenta es el ejecutor de consultas (*query executor*), ya que toda la aplicación se encuentra desarrollada en idioma inglés. Este componente se ubica en el centro de la pantalla y constituye el núcleo principal de interacción con el sistema.

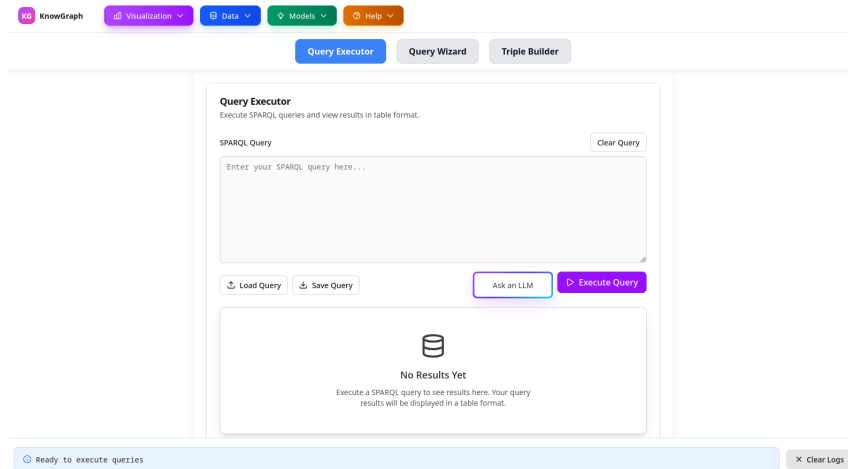


Figura 1: Pantalla de ejecución de consultas

En la parte superior de la interfaz se encuentra la barra de opciones, seguida de tres botones de control ubicados sobre el ejecutor de consultas. En la parte inferior de la pantalla se dispone la barra de mensajes, donde se muestran notificaciones y errores del sistema.

El ejecutor de consultas contiene un campo de texto editable en el cual el usuario puede ingresar una consulta escrita en lenguaje SPARQL. En caso de que el campo se encuentre vacío o la consulta contenga errores sintácticos o semánticos, el sistema notificará la situación mediante un mensaje de error en la barra de mensajes.

Sobre el campo de texto, hacia la derecha, se encuentra el botón *Clear Query*, cuya función es eliminar el contenido actual del campo de consultas, permitiendo al usuario comenzar una nueva consulta desde cero.

Debajo del campo de texto se presentan dos botones adicionales:

- *Load Query*: ubicado a la izquierda, abre un explorador de archivos y permite cargar el contenido de un archivo de texto directamente en el campo de consultas. Es importante destacar que el sistema no realiza una validación previa del contenido del archivo, por lo que se recomienda verificar que el texto corresponda a una consulta SPARQL válida antes de su ejecución.
- *Save Query*: ubicado a la derecha del botón anterior, permite guardar en un archivo el contenido actual del campo de consultas. De manera similar al botón de carga, este proceso no incluye una verificación automática de validez de la consulta, por lo que se sugiere comprobar su correcto funcionamiento antes de proceder con el guardado.

Siguiendo la misma línea de la interfaz, se encuentra el botón denominado *Ask LLM*. Al presionarlo, se despliega una ventana lateral en el sector derecho de la interfaz que habilita un canal de comunicación directo con un agente conversacional especializado en consultas SPARQL. Este agente está implementado utilizando la plataforma Botpress y es accedido mediante una API externa, por lo que su inicialización y tiempo de respuesta pueden variar en función de la disponibilidad del servicio y de la latencia de red. Durante este proceso, el usuario puede experimentar una breve demora hasta que el agente se encuentre completamente operativo.

La ventana del agente cuenta con un botón de cierre representado por una cruz, ubicado en la esquina superior derecha, que permite ocultarla y continuar utilizando el resto de las funcionalidades de la interfaz sin interrupciones. Mediante el campo de texto disponible, el usuario puede formular preguntas en lenguaje natural, solicitar explicaciones sobre consultas SPARQL existentes o requerir la generación automática de nuevas consultas a partir de una descripción del objetivo deseado. El agente responde de manera interactiva, facilitando la comprensión y el uso del lenguaje SPARQL, especialmente para usuarios con menor nivel de experiencia en este tipo de tecnologías.

En el extremo derecho de la barra de herramientas se encuentra el botón destinado a la ejecución de la consulta. Al activarlo, la consulta actualmente definida en el editor es enviada al servidor de consultas, el cual se encuentra implementado utilizando el framework Spring Boot e integra las funcionalidades provistas por Apache Jena para el procesamiento de consultas SPARQL. Este servidor es responsable de recibir la consulta, validarla, ejecutarla sobre el modelo de datos correspondiente y gestionar la obtención de los resultados.

Una vez finalizada la ejecución, los resultados son devueltos al cliente y presentados de forma estructurada en una tabla ubicada en la sección inferior de la interfaz. Dicha tabla se genera dinámicamente a partir de la consulta ejecutada: las variables especificadas en la cláusula **SELECT** de la consulta SPARQL se corresponden con las columnas de la tabla, mientras que cada fila representa una instancia recuperada como resultado de la ejecución.

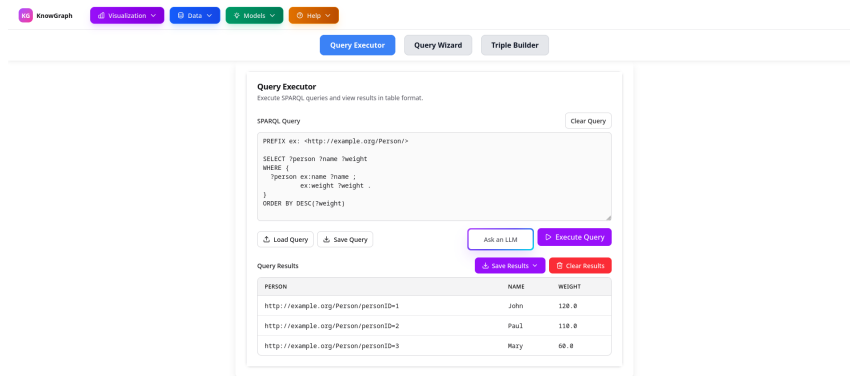


Figura 2: Pantalla luego de ejecutar una consulta

En función del volumen de datos devuelto por el servidor, la tabla puede extenderse tanto vertical como horizontalmente, habilitando mecanismos de desplazamiento que permiten al usuario navegar y visualizar la totalidad de los resultados obtenidos. Esta disposición facilita la exploración de los datos y el análisis de los resultados sin necesidad de abandonar la interfaz principal.

Una vez que los datos han sido procesados y mostrados, aparecerán dos botones en la parte superior derecha de la sección de resultados. El primero, denominado *Save Query*, despliega un menú que permite seleccionar el formato en el que se guardarán los resultados, ya sea *JSON* o *CSV*. Al elegir una de estas opciones, se abrirá el gestor de archivos del sistema, desde donde el usuario podrá almacenar los resultados en el formato seleccionado.

El segundo botón, denominado *Clear Results*, permite eliminar los resultados actualmente visibles en la interfaz, restableciendo el área de visualización.

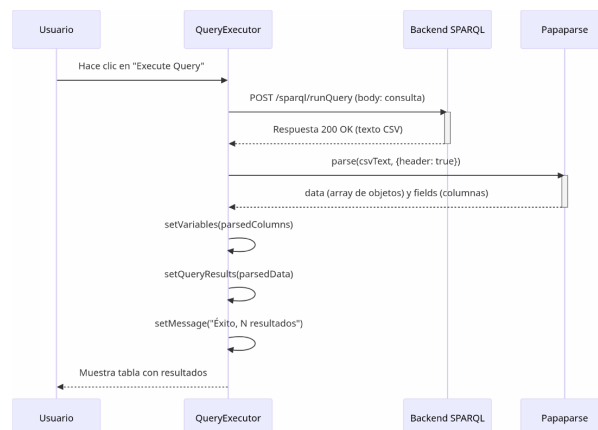


Figura 3: Diagrama secuencia ejecución de una consulta

En la parte superior derecha se encuentra el botón de *Help* que al seleccionarlo despliega un menú que propone dos ejemplos al hacer clic en alguno de ellos se cargara un conjunto de datos de ejemplo en el servidor y una consulta de ejemplo en el cliente para que ejecute. En el resto de la sección de funcionamiento de la aplicación se usarán estos conjuntos de datos como ejemplos y sus respectivas consultas asociadas.

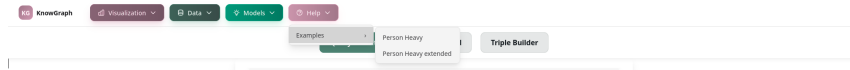


Figura 4: Ejemplos de conjuntos de datos

17. Sistema de Generación Visual de Consultas SPARQL

17.1. Introducción y Contexto

El desarrollo de consultas SPARQL (SPARQL Protocol and RDF Query Language) tradicionalmente requiere conocimiento especializado en la sintaxis del lenguaje, lo que representa una barrera de entrada para usuarios no técnicos. Para abordar este problema, se ha desarrollado un generador visual de consultas SPARQL que implementa un enfoque *wizard* (asistente) para la construcción de consultas. Este sistema permite a usuarios sin experiencia en SPARQL construir consultas complejas a través de una interfaz gráfica intuitiva, mientras genera código SPARQL válido y optimizado en segundo plano.

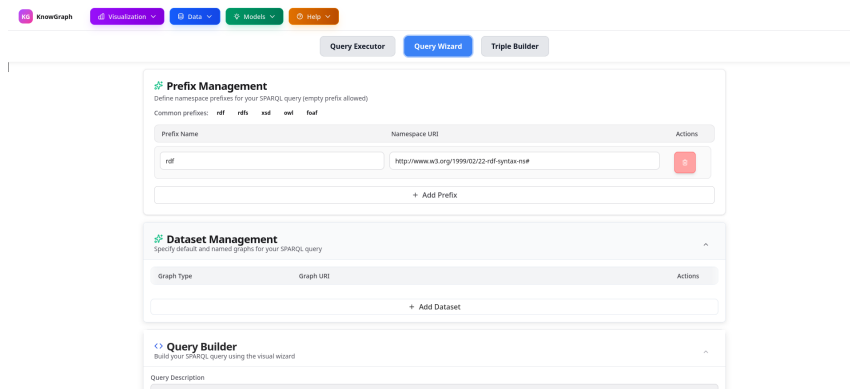


Figura 5: Gestor de prefijos en el generador de consultas

El sistema implementa el estándar SPARQL 1.1 completo, incluyendo características avanzadas como patrones de grafo, funciones de agregación, rutas de propiedades y consultas federadas, todo accesible a través de controles visuales.

17.2. Arquitectura del Sistema

El generador de consultas se compone de tres módulos principales interconectados:

1. *Gestor de Prefijos*: Administra las declaraciones de espacios de nombres (namespaces) utilizados en la consulta.
2. *Gestor de Conjuntos de Datos*: Configura los grafos RDF fuente (tanto grafos por defecto como nombrados).
3. *Constructor de Consultas*: Interfaz principal para la definición de patrones de triplas, filtros, funciones y cláusulas adicionales.

Estos módulos trabajan de manera coordinada para producir consultas SPARQL válidas que pueden ejecutarse directamente en cualquier endpoint SPARQL compatible.

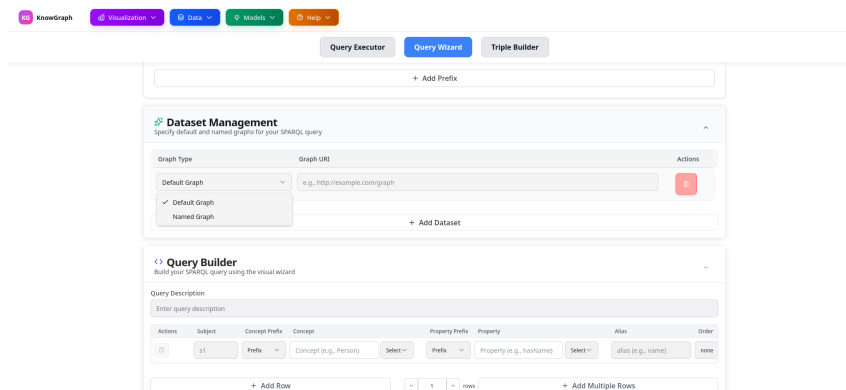


Figura 6: Gestor de conjuntos de datos

17.3. Flujo de Trabajo del Usuario

17.3.1. Paso 1: Configuración de Prefijos

Antes de construir la consulta, el usuario debe definir los espacios de nombres mediante el Gestor de Prefijos. Cada prefijo consiste en:

- *Nombre del Prefijo*: Identificador corto (ej. foaf)
- *URI del Espacio de Nombres*: URI completa del vocabulario (ej. <http://xmlns.com/foaf/0.1/>)

El sistema incluye prefijos comunes predefinidos (RDF, RDFS, XSD, OWL, FOAF) que pueden añadirse con un clic. También se permite definir un espacio de nombres por defecto utilizando un prefijo vacío.

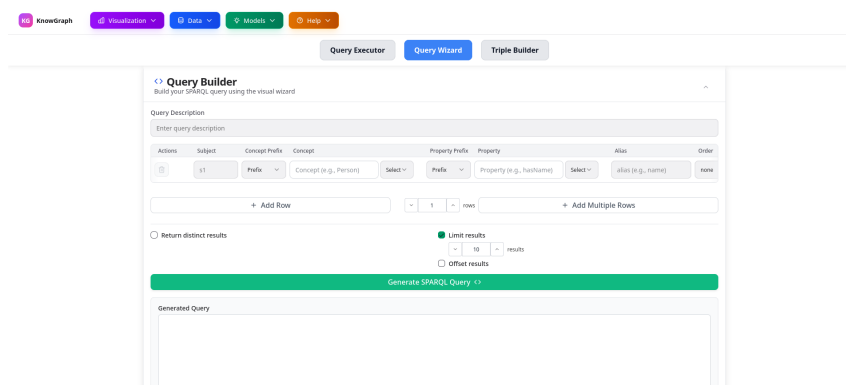


Figura 7: Constructor de consultas, interfaz principal

Ejemplo de configuración:

```
[H]
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
```

17.3.2. Paso 2: Especificación de Conjuntos de Datos

El Gestor de Conjuntos de Datos permite especificar las fuentes de datos RDF mediante cláusulas **FROM** y **FROM NAMED**:

- *Grafos por Defecto*: Fuentes principales de datos (cláusula **FROM**)
- *Grafos Nombrados*: Grafos específicos identificados por URI (cláusula **FROM NAMED**)

Esta configuración es opcional; si no se especifica, la consulta se ejecutará sobre el conjunto de datos por defecto del endpoint.

17.3.3. Paso 3: Construcción de la Consulta

La interfaz principal presenta una tabla donde cada fila representa un componente de la consulta. Cada columna corresponde a un elemento de la sintaxis SPARQL:

Tabla 1: Componentes de una fila de consulta en el constructor visual

<i>Componente</i>	<i>Descripción</i>	<i>Ejemplo</i>
Sujeto (<i>Subject</i>)	Variable que representa la entidad	?person
Concepto (<i>Concept</i>)	Tipo RDF de la entidad (opcional)	foaf:Person
Propiedad (<i>Property</i>)	Predicado RDF	foaf:name
Alias (<i>Alias</i>)	Variable para el valor resultante	?personName
Orden (<i>Order</i>)	Dirección de ordenamiento	Ascendente, Descendente
Visible (<i>Visible</i>)	Incluir variable en SELECT	Sí/No
Función (<i>Function</i>)	Función de agregación	COUNT, AVG, SUM, etc.
Operador (<i>Operator</i>)	Operador de filtro	=, >, contains, etc.
Valor (<i>Value</i>)	Valor para el filtro	"John", 30, etc.
Patrón (<i>Pattern</i>)	Tipo de patrón de grafo	Básico, Opcional, Unión, Menos
Grafo (<i>Graph</i>)	Grafo nombrado específico	<http://example.com/graph1>
Servicio (<i>Service</i>)	Endpoint SPARQL federado	<http://dbpedia.org/sparql>

Patrones de Triple Básicos: Para construir una tripla simple, el usuario completa:

- *Sujeto*: Nombre de variable (sin el prefijo ?)
- *Propiedad*: Predicado con prefijo opcional
- *Alias*: Variable para el objeto

Ejemplo: **?person foaf:name ?personName**

Entidades Tipadas: Para consultar entidades de un tipo específico, se completa adicionalmente el campo *Concepto*. El sistema genera automáticamente la tripla de tipo RDF: **?person rdf:type foaf:Person**

Filtros y Condiciones: Los filtros se aplican mediante la combinación de:

- Operador (comparación, coincidencia de cadenas, etc.)
- Valor (con detección automática de tipo: entero, decimal, booleano, string)

El sistema genera la sintaxis SPARQL apropiada con casting de tipos (**xsd:integer**, **xsd:string**, etc.).

Funciones de Agregación: Para consultas analíticas, se seleccionan funciones como:

- COUNT: Conteo de valores
- AVG: Promedio numérico
- SUM: Sumatoria
- GROUP_CONCAT: Concatenación de valores

Cuando se usa agregación, el sistema gestiona automáticamente las cláusulas **GROUP BY** y **HAVING**.

17.3.4. Paso 4: Configuración de Opciones Globales

Finalmente, el usuario configura opciones que aplican a toda la consulta:

- *DISTINCT*: Eliminar duplicados de los resultados
- *LIMIT*: Número máximo de resultados (paginación)
- *OFFSET*: Desplazamiento para paginación
- *ORDER BY*: Ordenamiento basado en variables visibles

17.4. Ejemplos de Uso

17.4.1. Ejemplo 1: Consulta Básica de Personas

Configuración:

- Prefijos: foaf, rdf
- Fila única:
 - Sujeto: person
 - Concepto: foaf:Person
 - Propiedad: foaf:name
 - Alias: personName
 - Visible: Sí
- Opciones: LIMIT = 10

Consulta Generada:

```
[H]
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT ?personName
WHERE {
    ?person rdf:type foaf:Person .
    ?person foaf:name ?personName .
}
LIMIT 10
```

17.4.2. Ejemplo 2: Consulta con Filtro y Agregación

Configuración:

- Fila 1: Sujeto=person, Propiedad=age, Alias=personAge, Función=Group by, Visible=Sí
- Fila 2: Sujeto=person, Propiedad=age, Alias=personAge, Función=Average, Resultado=avgAge, Visible=Sí
- Fila 3: Sujeto=person, Propiedad=department, Alias=dept, Operador==, Valor=.Engineering", Visible=Sí

Consulta Generada:

```
[H]
SELECT ?personAge (AVG(?personAge) AS ?avgAge) ?dept
WHERE {
    ?person age ?personAge .
    ?person department ?dept .
    FILTER (?dept = "Engineering")
}
GROUP BY ?personAge
```

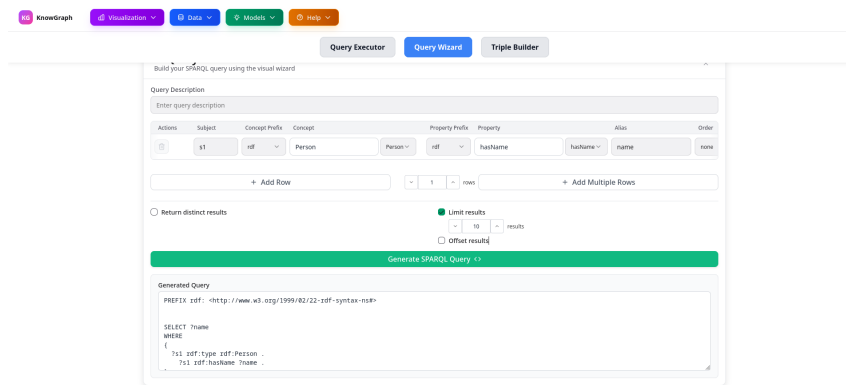


Figura 8: Ejemplo de consulta generada con el constructor

17.4.3. Ejemplo 3: Consulta Federada con Grafo Nombrado

Configuración:

- Dataset: Grafo nombrado <http://empresa.com/datos>
- Servicio: <http://dbpedia.org/sparql>
- Fila: Sujeto=person, Propiedad=foaf:name, Alias=name, Patrón=Optional

Consulta Generada:

```
[H]
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?name
FROM NAMED <http://empresa.com/datos>
WHERE {
  SERVICE <http://dbpedia.org/sparql> {
    OPTIONAL {
      ?person foaf:name ?name .
    }
  }
}
```

18. Constructor de triplas RDF para Consultas de Inserción

18.1. Introducción y Propósito

Mientras que la sección anterior abordaba la construcción de consultas SELECT para extraer información de grafos RDF, el *Triple Builder* (Constructor de triplas) se enfoca en la creación de consultas de inserción de datos. Este módulo permite construir consultas INSERT DATA de SPARQL, que son fundamentales para la carga y actualización de datos en repositorios RDF.

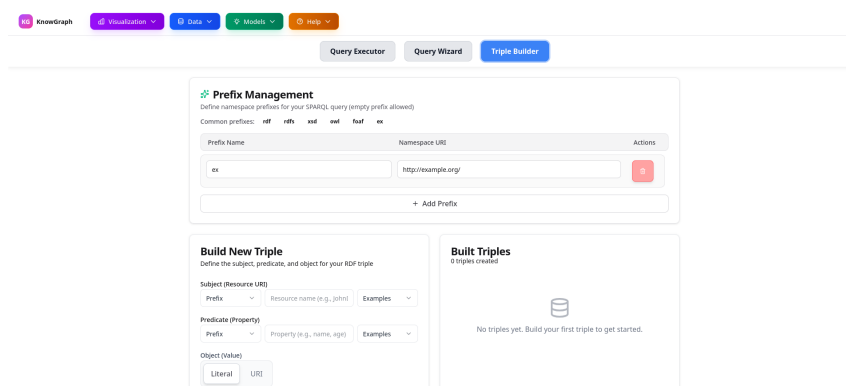


Figura 9: Imagen al entrar al constructor de triplas

El sistema está diseñado para usuarios que necesitan insertar nuevas triplas RDF en un grafo específico, proporcionando una interfaz intuitiva que abstrae la complejidad sintáctica de SPARQL y garantiza la validez de las triplas construidas.

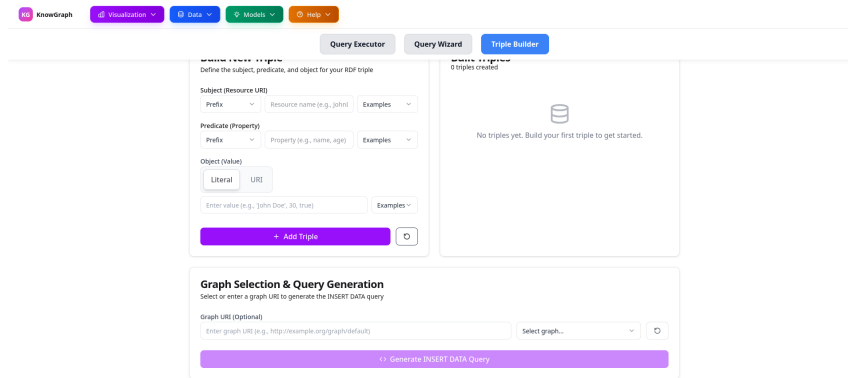


Figura 10: Constructor de triplas segunda parte

18.2. Arquitectura del Módulo

El Constructor de triplas implementa una arquitectura de tres componentes principales:

1. *Gestor de Prefijos*: Heredado del sistema anterior, permite definir espacios de nombres para abreviar URIs en las triplas.
2. *Constructor de triplas Individuales*: Interfaz para definir cada componente de una tripla RDF (sujeto, predicado, objeto).
3. *Administrador de triplas Construidas*: Panel donde se visualizan, organizan y gestionan todas las triplas antes de generar la consulta.
4. *Selector de Grafo Destino*: Mecanismo para especificar el grafo RDF donde se insertarán las triplas.

18.3. Flujo de Trabajo del Usuario

18.3.1. Paso 1: Configuración de Prefijos

Al igual que en el generador de consultas SELECT, el primer paso consiste en definir los prefijos necesarios. El sistema incluye prefijos comunes (RDF, RDFS, XSD, OWL, FOAF, EX) que pueden añadirse con un clic. Para cada prefijo se define:

- *Nombre*: Abreviatura (ej. foaf)
- *URI*: Identificador completo del espacio de nombres (ej. <http://xmlns.com/foaf/0.1/>)

Los prefijos permiten utilizar formas abreviadas en las triplas (ej. foaf:Person en lugar de <http://xmlns.com/foaf/0.1/Person>).

18.3.2. Paso 2: Construcción de triplas Individuales

Cada tripla RDF se construye mediante tres campos, cada uno con controles específicos: Para cada componente (sujeto, predicado, objeto), el usuario dispone de:

- *Selector de Prefijo*: Lista desplegable con los prefijos definidos
- *Campo de Texto*: Para ingresar el valor local del componente
- *Lista de Sugerencias*: Valores predefinidos para acelerar la entrada

Ejemplo de construcción:

- Sujeto: Prefijo = ex, Valor = JohnDoe
- Predicado: Prefijo = foaf, Valor = name
- Objeto: Prefijo = (ninguno), Valor = "John Doe"

Resulta en: ex:JohnDoe foaf:name "John Doe"

18.3.3. Paso 3: Gestión de Múltiples triplas

A medida que se construyen triplas, estas se agregan a un panel de visualización que muestra:

- Todas las triplas construidas en formato legible
- La estructura completa de cada tripla (sujeto, predicado, objeto)
- Opciones para eliminar triplas individuales
- Contador de triplas totales
- Botón para eliminar todas las triplas (reinicio completo)

Esta organización permite revisar y modificar el conjunto de triplas antes de generar la consulta final.

18.3.4. Paso 4: Selección del Grafo Destino

Las triplas RDF pueden insertarse en grafos específicos mediante la cláusula **GRAPH**. El sistema ofrece:

- *Campo de Texto*: Para ingresar manualmente la URI del grafo
- *Lista Desplegable*: Con grafos predefinidos (ej. <http://example.org/graph/people>)
- *Botón de Reinicio*: Para limpiar la selección actual

Si no se especifica un grafo, las triplas se insertan en el grafo por defecto del endpoint SPARQL.

18.3.5. Paso 5: Generación de la Consulta

Al hacer clic en "Generate INSERT DATA Query", el sistema:

1. Valida que exista al menos una tripla construida
2. Genera las declaraciones **PREFIX** basadas en los prefijos definidos
3. Construye la cláusula **INSERT DATA** con la estructura adecuada
4. Incluye la cláusula **GRAPH** si se ha especificado un grafo destino
5. Añade todas las triplas construidas en la sección de datos
6. Muestra la consulta completa en un área de texto de solo lectura
7. Envía automáticamente la consulta al ejecutor de SPARQL

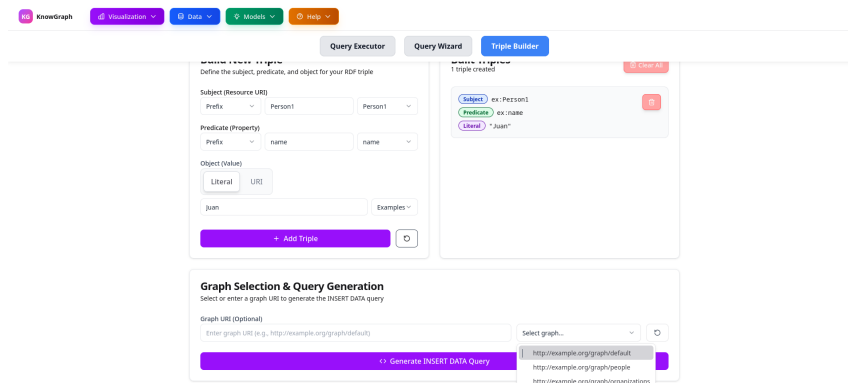


Figura 11: Tripla hecha con el constructor

18.4. Ejemplos de Uso

18.4.1. Ejemplo 1: Inserción de Datos de Persona

Configuración:

- Prefijos: foaf, ex
- Tripla 1:
 - Sujeto: ex:JohnDoe
 - Predicado: foaf:name
 - Objeto: "John Doe"
- Tripla 2:
 - Sujeto: ex:JohnDoe
 - Predicado: foaf:age
 - Objeto: "30"^^xsd:integer
- Grafo: http://example.org/graph/people

Consulta Generada:

```
[H]
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX ex: <http://example.org/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

INSERT DATA {
  GRAPH <http://example.org/graph/people> {
    ex:JohnDoe foaf:name "John Doe" .
    ex:JohnDoe foaf:age "30"^^xsd:integer .
  }
}
```

18.4.2. Ejemplo 2: Inserción de Relaciones Organizacionales

Configuración:

- Prefijos: ex, org
- Tripla 1:
 - Sujeto: ex:JohnDoe
 - Predicado: ex:worksFor
 - Objeto: ex:CompanyXYZ
- Tripla 2:
 - Sujeto: ex:CompanyXYZ
 - Predicado: ex:hasName
 - Objeto: Company XYZ Inc."
- Grafo: (ninguno - grafo por defecto)

Consulta Generada:

```
[H]
PREFIX ex: <http://example.org/>
PREFIX org: <http://example.org/ontology/org#>

INSERT DATA {
  ex:JohnDoe ex:worksFor ex:CompanyXYZ .
  ex:CompanyXYZ ex:hasName "Company XYZ Inc." .
}
```

18.4.3. Ejemplo 3: Inserción Masiva con Tipado

Configuración:

- Prefijos: `rdf`, `foaf`, `ex`
- Múltiples triplas para diferentes personas
- Cada persona incluye: tipo (`foaf:Person`), nombre, email
- Grafo: `http://example.org/graph/employees`

Consulta Generada (fragmento):

```
[H]
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX ex: <http://example.org/>

INSERT DATA {
  GRAPH <http://example.org/graph/employees> {
    ex:Person1 rdf:type foaf:Person .
    ex:Person1 foaf:name "Alice Smith" .
    ex:Person1 foaf:mbox "alice@example.org" .

    ex:Person2 rdf:type foaf:Person .
    ex:Person2 foaf:name "Bob Johnson" .
    ex:Person2 foaf:mbox "bob@example.org" .

    # ... más triplas
  }
}
```

18.5. Características Avanzadas

Detección Automática de Tipos de Datos: El sistema detecta automáticamente el tipo de dato de los objetos literales:

- Números enteros: `"30"8sd:integer`
- Números decimales: `"30.5"8sd:decimal`
- Valores booleanos: `"true"8sd:boolean`
- Cadenas de texto: `"Texto literal"`

Sugerencias Contextuales:

- Lista de propiedades comunes (`hasName`, `worksFor`, etc.)
- Variables predefinidas (`?person`, `?org`, etc.)
- Grafos comúnmente utilizados

Validación en Tiempo Real:

- Verificación de que los tres componentes estén completos
- Validación de formato de URIs en los prefijos
- Comprobación de que al menos una tripla esté presente para generar la consulta

18.6. Integración con el Sistema General

El Constructor de triplas se integra con el sistema general de gestión de consultas SPARQL de las siguientes maneras:

1. *Compartición de Prefijos*: Los prefijos definidos en este módulo pueden ser utilizados por otros componentes del sistema.
2. *Flujo Unificado*: La consulta generada se envía automáticamente al ejecutor de SPARQL.
3. *Consistencia de Interfaz*: Mantiene el mismo diseño visual y patrones de interacción que el generador de consultas SELECT.
4. *Mensajería Unificada*: Utiliza el mismo sistema de notificaciones para informar sobre el estado de las operaciones.

19. Visualización de Datos en KnowGraph

Para visualizar datos en la aplicación KnowGraph, es necesario realizar previamente una consulta sobre la base de datos. A partir de los resultados obtenidos, es posible generar visualizaciones utilizando dos métodos distintos, ambos con soporte para representaciones en dos (2D) y tres dimensiones (3D). Estos métodos se encuentran disponibles en el menú desplegable del botón *Visualization*.

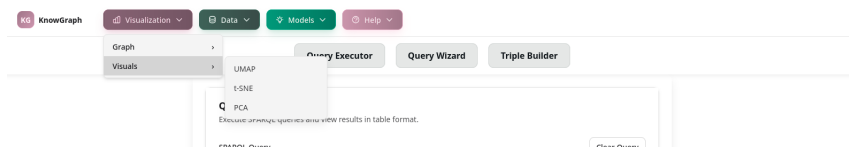


Figura 12: Botón de visualización, menú de reducción dimensional seleccionado

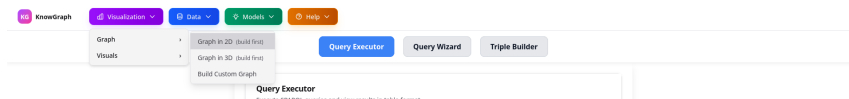


Figura 13: Botón de visualización, menú de reducción generar grafo seleccionado

19.1. Visualización en forma de grafo

Dentro del menú de visualización, la sección *Graph* despliega tres opciones: *Graph in 2D*, *Graph in 3D* y *Build Custom Graph*. Para visualizar un grafo, el primer paso consiste en su construcción. En caso de que aún no se haya creado ningún grafo, al seleccionar cualquiera de las opciones *Graph in 2D* o *Graph in 3D* aparecerá un mensaje lateral indicando la necesidad de construirlo (*Build first*). Al presionar dicho botón, se abrirá un panel de configuración.

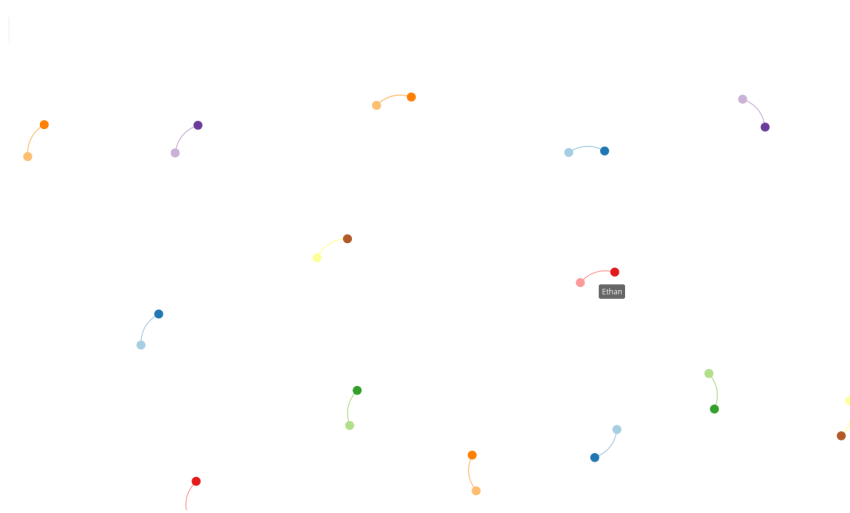


Figura 14: Ejemplo de grafo en 2D



Figura 15: Ejemplo de grafo en 3D

En este panel se presentan las variables obtenidas en la consulta, organizadas en dos secciones: *Node Variables*, donde se seleccionan las variables que actuarán como nodos, y *Edge Variables*, destinadas a las variables que representarán los arcos del grafo. La selección se realiza de manera sencilla mediante la marcación de las variables correspondientes en cada sección.

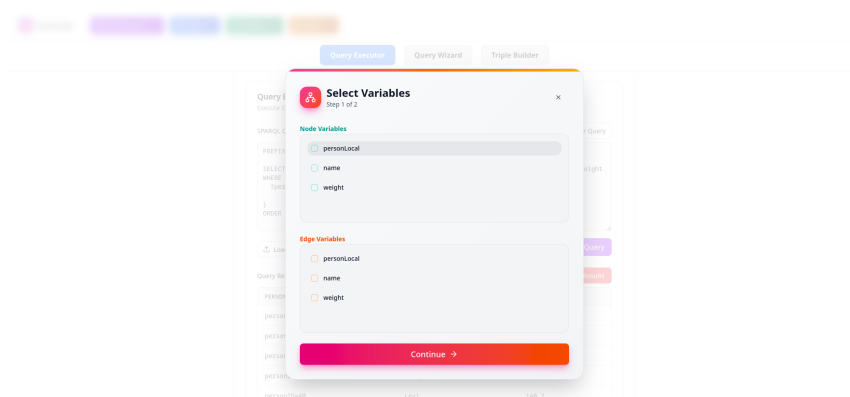


Figura 16: Primera paso constructor de grafos

Una vez seleccionadas las variables, se procede a definir las relaciones entre los nodos. Para cada par de variables de nodo se debe especificar el tipo de conexión, tras lo cual se habilita la selección de las variables de arco asociadas, siguiendo el mismo mecanismo de marcado. Finalizada esta configuración, al presionar el botón de confirmación se genera el grafo en el formato elegido (2D o 3D). Al interactuar con los nodos o arcos del grafo, es posible visualizar los valores asociados a cada uno de ellos. La opción *Build Custom Graph* permite construir un nuevo grafo una vez que

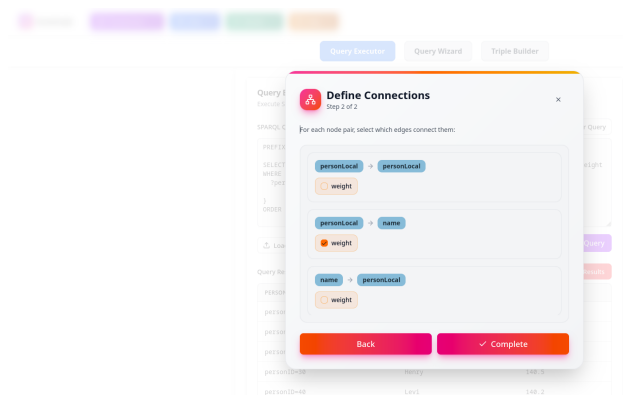


Figura 17: Segundo paso constructor de grafos

ya existe uno visualizado. Al utilizar esta opción, el grafo actual es reemplazado por uno nuevo, siguiendo el mismo proceso de construcción que en el caso inicial.

19.2. Visualización mediante reducción dimensional

Para la visualización mediante técnicas de reducción dimensional, se debe acceder a la sección *Visuals* del menú desplegable. Allí se encuentran disponibles tres métodos: *UMAP*, *t-SNE* y *PCA*. Si bien estos métodos difieren en su enfoque algorítmico, el proceso de creación de la visualización ha sido unificado para simplificar su uso.

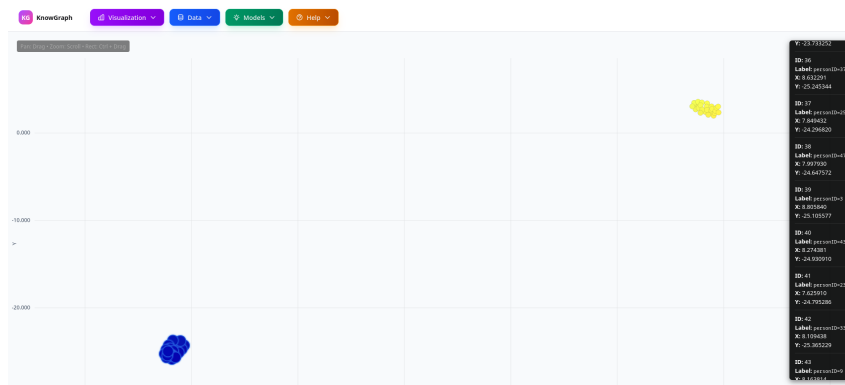


Figura 18: Ejemplo de umap en 2D

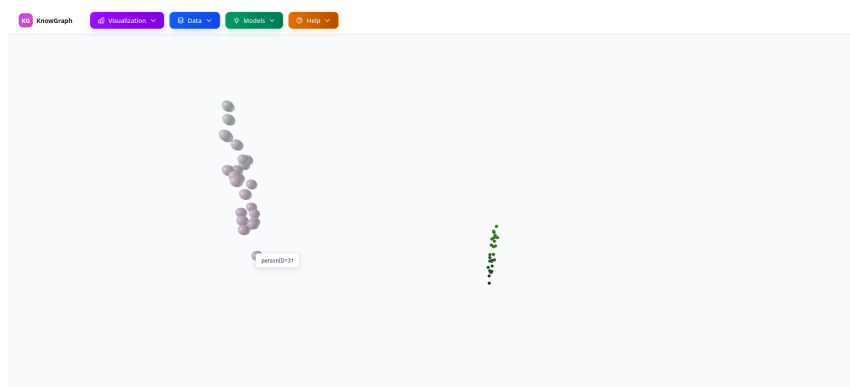


Figura 19: Ejemplo de umap en 3D

Al seleccionar cualquiera de estos métodos, se abre un panel en el que se pueden elegir las variables que serán utilizadas para realizar la reducción dimensional. Para generar una visualización en dos dimensiones se requiere seleccionar al menos dos variables, mientras que para una visualización

en tres dimensiones se deben seleccionar tres o más. La cantidad de dimensiones se define mediante un menú desplegable ubicado en la esquina superior derecha del panel.

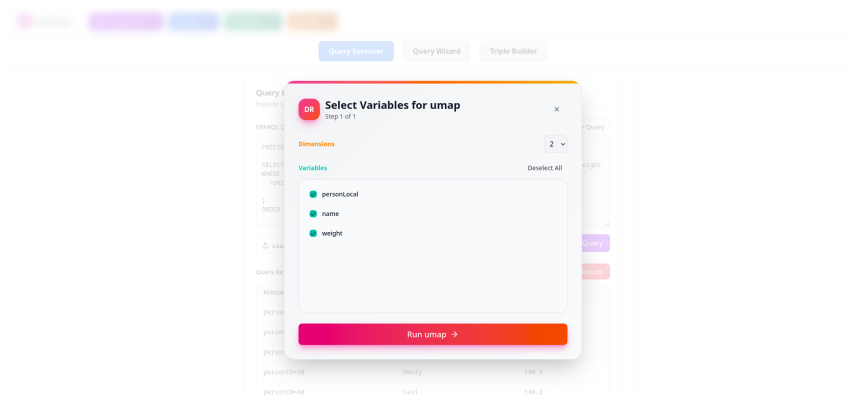


Figura 20: Selector de variables para visualizacion

Una vez seleccionadas las variables, se puede iniciar el proceso presionando el botón correspondiente, lo que conduce a una pantalla de carga hasta que la visualización es generada. Finalmente, los datos se muestran en forma de puntos en el espacio reducido. El panel también incluye un botón *Select All*, que permite seleccionar todas las variables disponibles de manera automática.

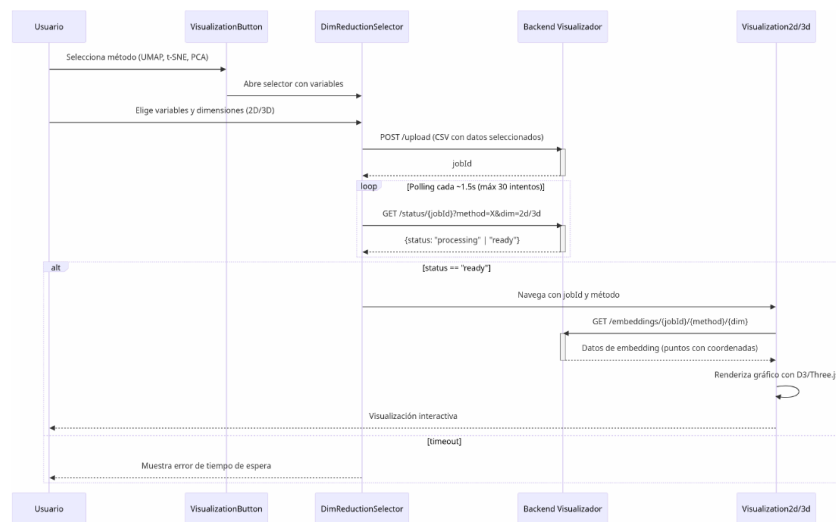


Figura 21: Diagrama de secuencia para usar el microservicio de importacion

20. Carga de datasets en KnowGraph

Para cargar datos en la aplicación KnowGraph se sigue un proceso sencillo a través del botón *Data*. Al presionarlo, se despliega un menú en el que las primeras tres opciones permiten la carga de datos en el servidor, mientras que la última opción se utiliza para eliminar los datos actualmente almacenados.

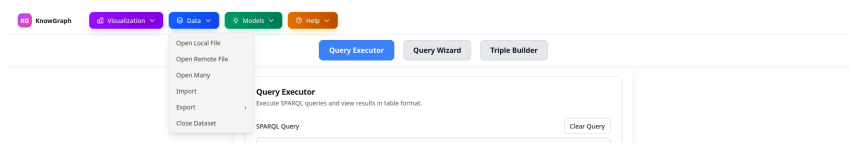


Figura 22: Botón de manejo de datos

La opción *Open local file* abre el gestor de archivos del sistema, desde donde se puede seleccionar un archivo en alguno de los formatos soportados: *TURTLE* (.ttl), *N-Triples* (.nt), *N3* (.n3),

RDF/XML (.owl), *JSON-LD*, *TriX* y *N-Quads*. En caso de seleccionar un archivo con un formato no reconocido, el servidor generará un error que se mostrará en la parte inferior de la interfaz.

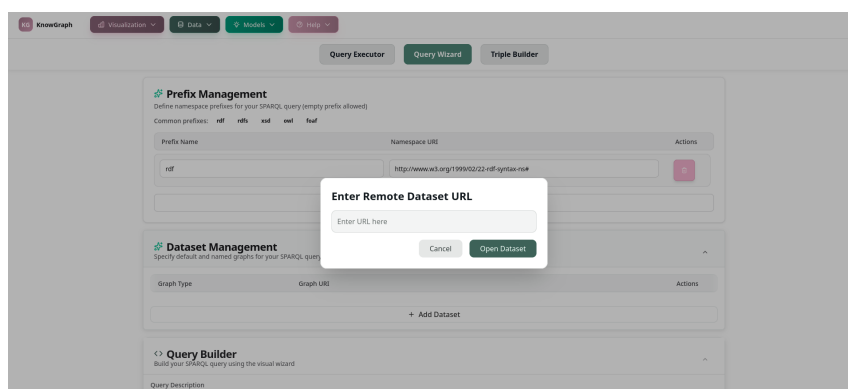


Figura 23: Menu para carga de datos remota

La opción *Open remote* permite cargar datasets desde un repositorio remoto. Al seleccionarla, se despliega un panel en el que se debe ingresar la URL correspondiente. Si el enlace proporcionado no es válido o no puede ser procesado por el servidor, se mostrará un mensaje de error.

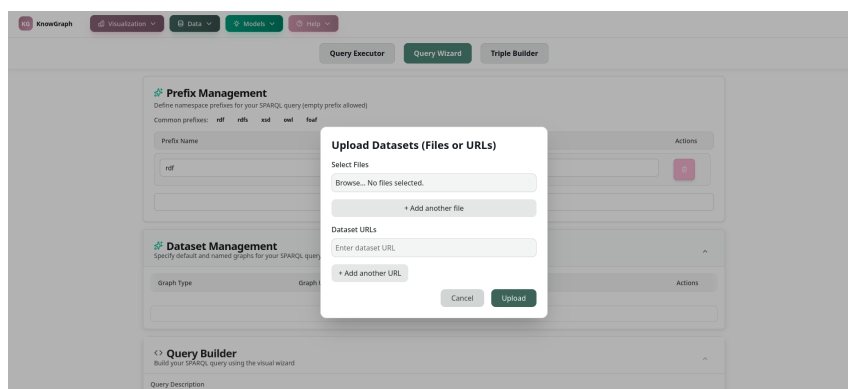


Figura 24: Menu de carga de multiples archivos

Por último, la opción *Open many* combina ambas modalidades de carga. En la parte superior del panel se pueden seleccionar archivos locales, mientras que en la parte inferior se pueden especificar enlaces a archivos remotos. Una vez seleccionados todos los datasets, al presionar el botón de apertura estos se cargan en el servidor y se combinan con los datos previamente existentes. A diferencia de esta opción, las modalidades *Open local file* y *Open remote* reemplazan completamente los datos que ya estaban cargados en el servidor.

El botón *Close dataset* elimina de la memoria del servidor todos los datos cargados, permitiendo iniciar una nueva carga desde cero.

21. Importación de datos

En el mismo menú desplegable utilizado para la carga de datasets se encuentra la opción *Import*, destinada a la importación de datos desde formatos no RDF. Al seleccionarla, se accede a un nuevo menú en el que es posible elegir el tipo de archivo a importar entre los siguientes: *CSV*, *Excel*, *JSON* y *XML*.

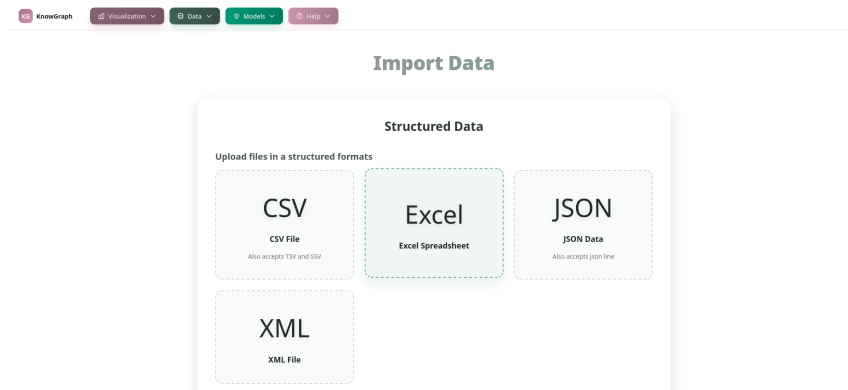


Figura 25: Menú importación de datos parte 1

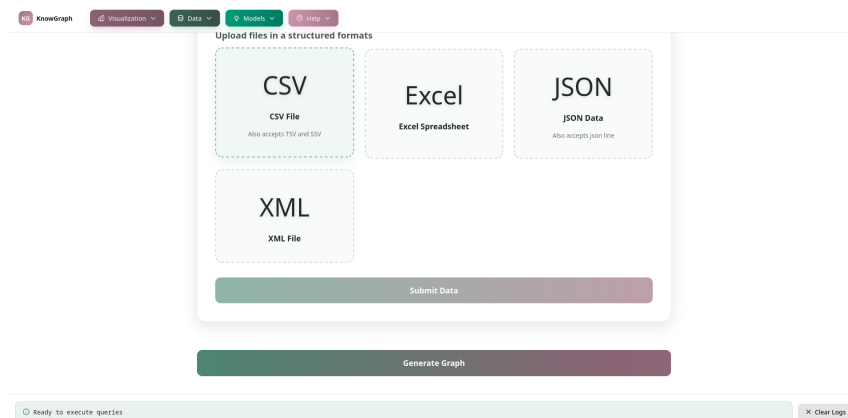


Figura 26: Menú importación de datos parte 2

Al presionar el botón correspondiente al formato deseado, se abre el gestor de archivos para seleccionar el archivo, cuyo nombre y tipo se mostrarán en la parte inferior de la interfaz. Una vez seleccionados todos los archivos a importar, se debe presionar el botón *Submit data* para enviarlos al servidor.

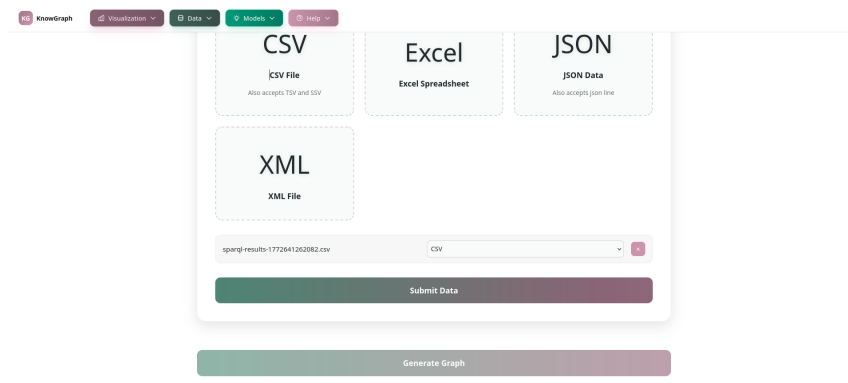


Figura 27: Menú importación de datos con un archivo para subir al servidor

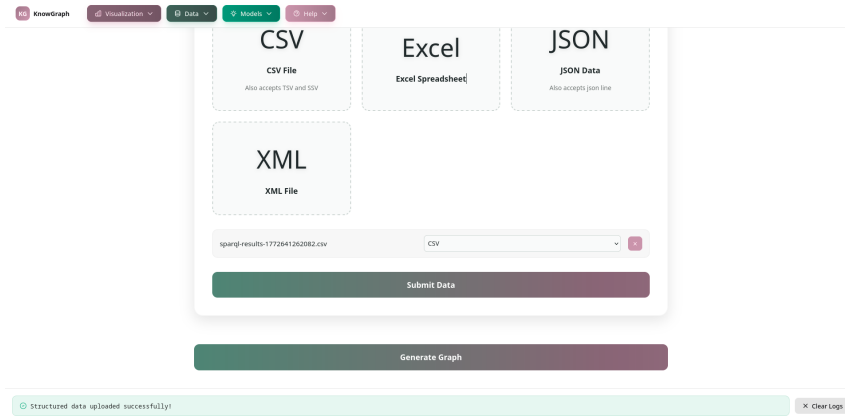


Figura 28: Menu de importación listo para generar un grafo

Si ocurre algún error durante el proceso, se mostrará un mensaje descriptivo en la parte inferior de la pantalla. En caso contrario, se presentará un mensaje de confirmación indicando que la importación fue exitosa. Posteriormente, al presionar el botón de generación de grafo, el servidor construirá un grafo a partir de los datos importados y lo exportará en formato *Turtle*. Finalmente, se abrirá el gestor de archivos para que el usuario pueda guardar el archivo generado en su computadora.

Para regresar al menú principal de la aplicación, se debe presionar el botón con el logotipo de KnowGraph, identificado con las siglas *KG*.

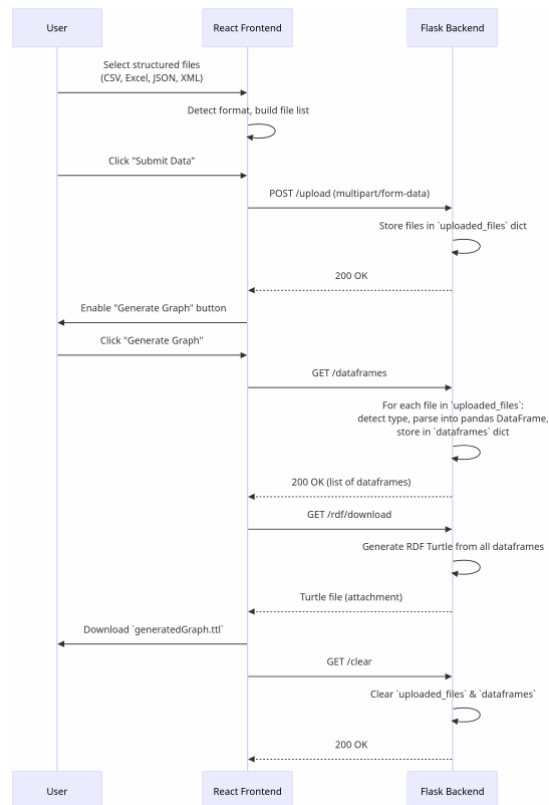


Figura 29: Diagrama de secuencia para usar el microservicio de importacion

22. Exportación de datos

En el menú desplegable del botón *Data* se encuentra la opción *Export*, que permite transformar un dataset cargado en el servidor de un formato a otro.

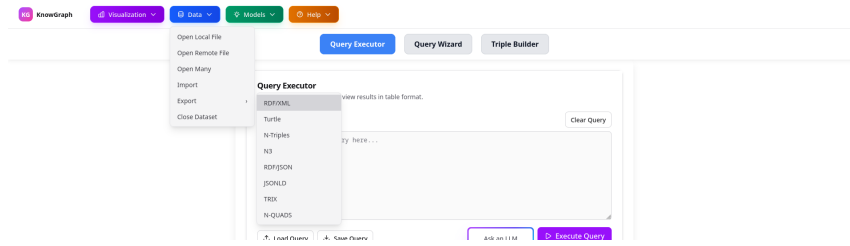


Figura 30: Distintos formatos para exportar un grafo cargado en el servidor

Una vez que uno o varios conjuntos de datos han sido cargados en el servidor, estos pueden exportarse seleccionando el formato de salida deseado. El dataset resultante se genera automáticamente y puede ser descargado y almacenado en la computadora del usuario.

23. Razonamiento sobre datos

La aplicación KnowGraph permite realizar razonamiento semántico sobre los datos cargados a través del botón *Models* de la interfaz. Al posicionarse sobre este botón, se despliega un menú con dos opciones principales: la creación de un *Ontology Model* y la creación de un *Inference Model*. Cada una de estas opciones cuenta con su propio conjunto de configuraciones, que permiten seleccionar tanto el tipo de modelo como el razonador a utilizar.

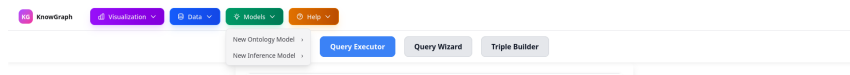


Figura 31: Menu boton de modelos

23.1. Modelo de ontología

Una ontología es una representación formal y estructurada del conocimiento de un dominio específico. Define los conceptos, relaciones, propiedades y restricciones que caracterizan dicho dominio, permitiendo una descripción precisa y no ambigua de la información.

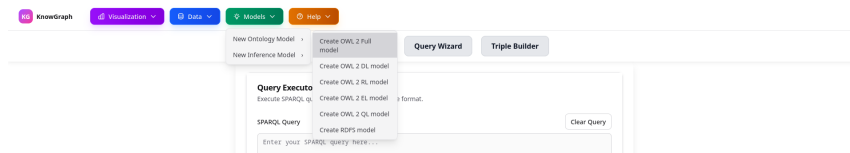


Figura 32: Distintos modelos de ontología soportados por Apache Jena

En el ámbito de la informática y de la Web Semántica, las ontologías se emplean para que los sistemas puedan interpretar, compartir y razonar sobre los datos, favoreciendo la interoperabilidad entre aplicaciones y la reutilización del conocimiento. Estas ontologías suelen expresarse mediante lenguajes formales como *RDF*, *RDFS* y *OWL*.

En KnowGraph, al seleccionar la opción de *Ontology Model*, se crea un nuevo modelo vacío desde cero, basado en el tipo de ontología seleccionado. Los modelos y ontologías disponibles son los provistos por el framework *Apache Jena*, lo que garantiza compatibilidad con estándares ampliamente utilizados en la Web Semántica. Este tipo de modelo se utiliza principalmente para definir esquemas conceptuales y vocabularios, sin necesidad de contar inicialmente con datos instanciados.

23.2. Modelo de inferencia

El *Inference Model* permite realizar razonamiento automático sobre los datos previamente cargados en el servidor. Al seleccionar un modelo de inferencia, la aplicación genera un nuevo modelo que incluye tanto los datos originales como todas las afirmaciones inferidas a partir de ellos, según las reglas del razonador seleccionado.

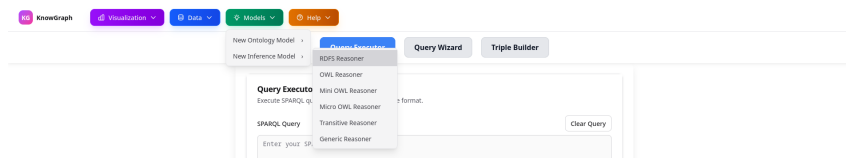


Figura 33: Distintos modelos de inferencia soportados por Apache Jena

Apache Jena ofrece distintos tipos de razonadores estándar, tales como razonadores basados en *RDFS* y *OWL*, los cuales permiten inferir nueva información a partir de jerarquías de clases, relaciones de subclase, propiedades y restricciones definidas en la ontología. Además, la aplicación incluye un razonador genérico, que permite al usuario definir un conjunto personalizado de reglas de inferencia. Estas reglas especifican patrones lógicos que, al cumplirse, generan nuevas afirmaciones, proporcionando mayor flexibilidad y control sobre el proceso de razonamiento.

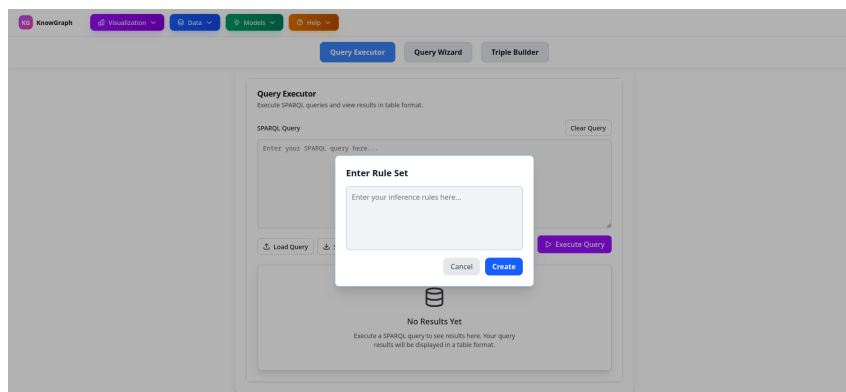


Figura 34: Gestor de reglas para modelo de inferencia genérico

El uso de modelos de inferencia resulta fundamental para enriquecer los datos, descubrir conocimiento implícito y mejorar la capacidad de consulta y análisis dentro de la aplicación.

24. Documentación para desarrolladores

Este documento constituye la documentación técnica completa de un sistema dockerizado para la gestión, consulta y visualización de grafos de conocimiento. El sistema se compone de un *frontend* construido con Vite, React y TypeScript; un *backend en Java* que utiliza Apache Jena para el razonamiento sobre grafos y Spring Boot para la exposición de servicios REST; y un *backend en Python* basado en Flask/Gunicorn, responsable de la importación de datos y de las transformaciones para su visualización. Esta documentación detalla la arquitectura, los componentes principales, los flujos de datos y las interacciones entre módulos.

25. Introducción y Arquitectura General

La aplicación es un sistema integral diseñado para trabajar con grafos de conocimiento (Knowledge Graphs). Sigue una arquitectura de microservicios contenerizados (Docker) que separa las responsabilidades en tres capas principales:

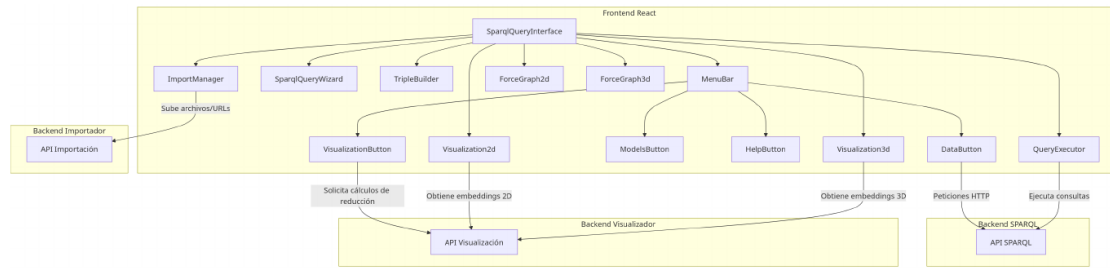


Figura 35: Arquitectura del Sistema

- **Frontend:** Aplicación web interactiva construida con el framework Vite, utilizando React y TypeScript. Sirve una interfaz de usuario estática mediante `http-server`.
- **Backend Java:** Servicio principal que maneja toda la lógica de ontologías y consultas. Utiliza *Apache Jena* para las operaciones sobre el grafo RDF y el razonamiento, y *Spring Boot* para estructurar la aplicación y exponer una API REST.
- **Backend Python:** Conjunto de dos microservicios que amplían la funcionalidad:
 - *Importer:* Gestiona la importación de datos desde formatos estructurados (CSV, JSON, Excel, XML) y su conversión a RDF.
 - *Visualizer:* Realiza transformaciones de datos, específicamente reducción dimensional (PCA, t-SNE, UMAP), para habilitar visualizaciones avanzadas.

Ambos servicios están implementados con *Flask* y desplegados con *Gunicorn*.

La comunicación entre el frontend y los backends se realiza exclusivamente a través de peticiones HTTP/REST.

26. Backend Java (Spring Boot y Apache Jena)

Esta sección describe el núcleo del sistema, encargado del almacenamiento, razonamiento y consulta del grafo de conocimiento.

26.1. Apache Jena: Gestión del Grafo y Ejecución SPARQL

Jena es la biblioteca principal para manipular el modelo RDF/OWL y ejecutar consultas SPARQL. Las clases diseñadas para esta tarea siguen patrones que favorecen la extensibilidad y la claridad.

26.1.1. SparqlQueryResult y SparqlQueryResultData

SparqlQueryResultData: Es un objeto de transferencia de datos (DTO) que encapsula los metadatos y resultados de cualquier consulta SPARQL (tipo, variables, filas, modelo RDF, resultado booleano o mensaje de error).

SparqlQueryResult: Es el contenedor unificado para todos los tipos de resultados. Ofrece métodos *factory* (e.g., `forSelect()`, `forConstruct()`) y métodos de conversión automática a formatos de salida como CSV o JSON (`getResultAsStringObject()`). Maneja de forma transparente los diferentes tipos de consulta (SELECT, CONSTRUCT, ASK, DESCRIBE) y los errores.

26.1.2. Fábrica y Ejecutores de Consultas (Patrón Factory)

Este patrón centraliza la lógica de ejecución:

```
SparqlQueryExecutorFactory.getExecutor(Query query)
```

- *SparqlQueryExecutorFactory:* Analiza el tipo de consulta SPARQL y devuelve la instancia adecuada de un ejecutor concreto.

- *SparqlQueryExecutor*: Interfaz que define el contrato común (`execute()`).
- *Ejecutores Concretos*:
 - *SelectQueryExecutor*: Para consultas SELECT (resultados tabulares).
 - *ConstructQueryExecutor* y *DescribeQueryExecutor*: Para consultas que generan un nuevo modelo RDF.
 - *AskQueryExecutor*: Para consultas ASK (resultado booleano).

El flujo de una consulta es: Consulta SPARQL $\rightarrow$ *SparqlQueryExecutorFactory* $\rightarrow$ Ejecutor Específico $\rightarrow$ Resultado Jena $\rightarrow$ *SparqlQueryResult* $\rightarrow$ Formato final (CSV/JSON).

26.1.3. OntoManipulator e InferenceManipulator

OntoManipulator: Se encarga de crear y gestionar modelos ontológicos (OWL2, RDFS) con capacidades de inferencia integrada. Proporciona acceso a diferentes vistas del modelo: el modelo base, el modelo inferencial y el modelo ontológico completo. Soporta varios perfiles OWL2 (DL, Full, EL, QL, RL) y RDFS.

InferenceManipulator: Gestiona la creación de modelos de inferencia sobre datos RDF existentes. Permite usar *reasoners* predefinidos (RDFS, OWL) o reglas personalizadas escritas en sintaxis Jena. Ofrece funcionalidades avanzadas como validación de consistencia, listado de triplas inferidas y explicación de derivaciones (`explainDerivation`).

26.1.4. SparqlService

Es el servicio central (anotado con `@Service`) que orquesta las operaciones principales:

- *Ejecución de consultas*: Expone el método `runSparqlQuery(String)` que sigue el flujo descrito anteriormente.
- *Carga de datos*: Métodos para cargar o añadir datos RDF al dataset central desde archivos, URLs o streams, soportando múltiples formatos (RDF/XML, Turtle, JSON-LD, etc.).
- *Exportación*: Permite exportar el grafo completo a diversos formatos de serialización RDF.
- *Gestión del Dataset*: Mantiene el dataset Jena que contiene todos los grafos cargados.

26.2. Spring Boot: API REST y Configuración

Spring Boot se utiliza para la configuración automática, la inyección de dependencias y la creación de la API REST.

26.2.1. ApplicationRunner

Clase principal que arranca la aplicación. Implementa `CommandLineRunner` y está anotada con `@Configuration`, `@EnableAutoConfiguration` y `@ComponentScan`.

26.2.2. Controladores REST

SparqlController (`@RestController`): Expone los endpoints principales en `/sparql/*`.

- POST `/sparql/runQuery`: Ejecuta una consulta SPARQL.
- POST `/sparql/loadDatasetFrom*`: Carga datos desde URL o archivo.
- GET `/sparql/getExport`: Exporta el dataset.
- DELETE `/sparql/clearDataset`: Limpia el dataset.
- Endpoints para cargar ejemplos predefinidos.

OntModelController: Expone GET `/ontmodel/createModel` para crear modelos ontológicos de diversos tipos, que luego se cargan en el *SparqlService*.

InfModelController: Expone endpoints en `/infmodel/*` para operaciones de inferencia: crear modelos con *reasoners* predefinidos o reglas genéricas, validar consistencia, listar triplas inferidas y explicar derivaciones.

26.3. Maven

Se utiliza como herramienta de gestión de dependencias y construcción. El archivo `pom.xml` define las dependencias clave (Spring Boot, Apache Jena) y los plugins para empaquetar la aplicación. Comandos esenciales:

- `mvn clean package`: Genera el archivo JAR ejecutable.
- `mvn spring-boot:run`: Inicia la aplicación localmente.

27. Backend Python (Microservicios)

Python se emplea para dos microservicios específicos, desacoplados del núcleo Java.

27.1. Importer (`importer.py`)

Servicio Flask (puerto 5001) para importar datos estructurados y convertirlos a RDF.

27.1.1. Propósito y Funcionalidad

- Carga archivos en memoria desde formatos como CSV, JSON, Excel y XML.
- Convierte cada archivo en uno o varios DataFrames de Pandas.
- Transforma los DataFrames a representaciones RDF en formato Turtle.
- Gestiona el ciclo de vida de los archivos subidos.

27.1.2. Endpoints Principales

```
[H]
POST    /upload          # Subir archivos
GET     /dataframes    # Convertir a DataFrames
GET     /list        # Listar archivos
GET     /use/<file>   # Información de un archivo
GET     /rdf_all     # Generar RDF de todos los DataFrames
GET     /rdf/download # Descargar grafo RDF completo
```

27.1.3. Flujo de Trabajo

1. *Carga*: Los archivos se suben y almacenan en memoria como bytes. 2. *Conversión*: Se detecta el tipo por extensión y se parsea con la librería adecuada (Pandas). Para CSV, se detecta automáticamente el delimitador. 3. *Exportación RDF*: Cada fila del DataFrame se convierte en un sujeto con URI única (`http://example.org/filename_index`). Cada columna no vacía se convierte en un predicado, y su valor en un objeto literal. Se genera un documento Turtle unificado.

27.2. Visualizer (`visualizer.py`)

Servicio Flask (puerto 5000) para reducción dimensional y generación de *embeddings* visualizables.

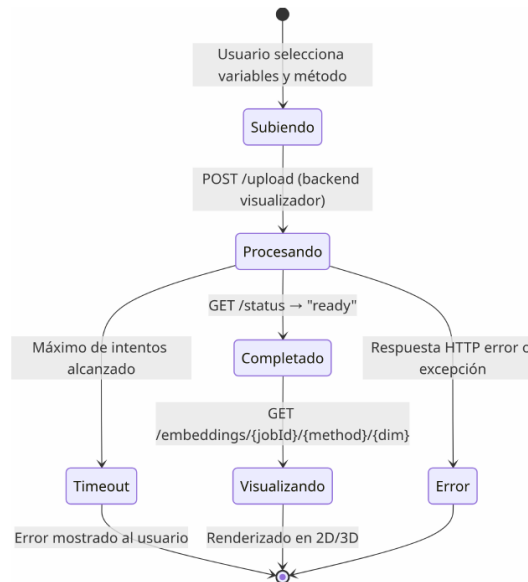


Figura 36: Diagrama de estados del servicio de visualización

27.2.1. Propósito y Funcionalidad

- Preprocesa datasets (CSV) para análisis.
- Aplica algoritmos de reducción dimensional: PCA, t-SNE y UMAP.
- Genera *embeddings* en 2D y 3D bajo demanda (*lazy evaluation*).

27.2.2. Algoritmos y Preprocesamiento

- *Preprocesamiento*: Detección automática de columna de etiqueta. Las columnas se clasifican en:
 - Numéricas: Estandarizadas con **StandardScaler**.
 - Categóricas (baja cardinalidad): Codificación one-hot.
 - Texto: Vectorización TF-IDF.
- *Algoritmos*: Configuraciones por defecto para PCA, t-SNE (perplejidad: 30) y UMAP (vecinos: 15, distancia mínima: 0.1).

27.2.3. Flujo de la API

1. **POST /upload**: El cliente envía un CSV y parámetros. El servidor preprocesa los datos, genera un `job_id` y los almacena, pero *no calcula embeddings aún*.
2. **GET /embeddings/job_id/método/dims**: Primera solicitud para un método y dimensión específicos (e.g., `tsne_3d`) dispara el cálculo, que se cachea. Solicitudes posteriores devuelven el resultado cacheado.

27.2.4. Estructura de Almacenamiento

Los resultados se guardan en un diccionario global **RESULTS** indexado por `job_id`, que contiene los datos preprocesados, parámetros, la columna de etiqueta y un caché de embeddings ya calculados.

28. Frontend (TypeScript, React y Vite)

Aplicación de una sola página (SPA) que consume las APIs de los backends.

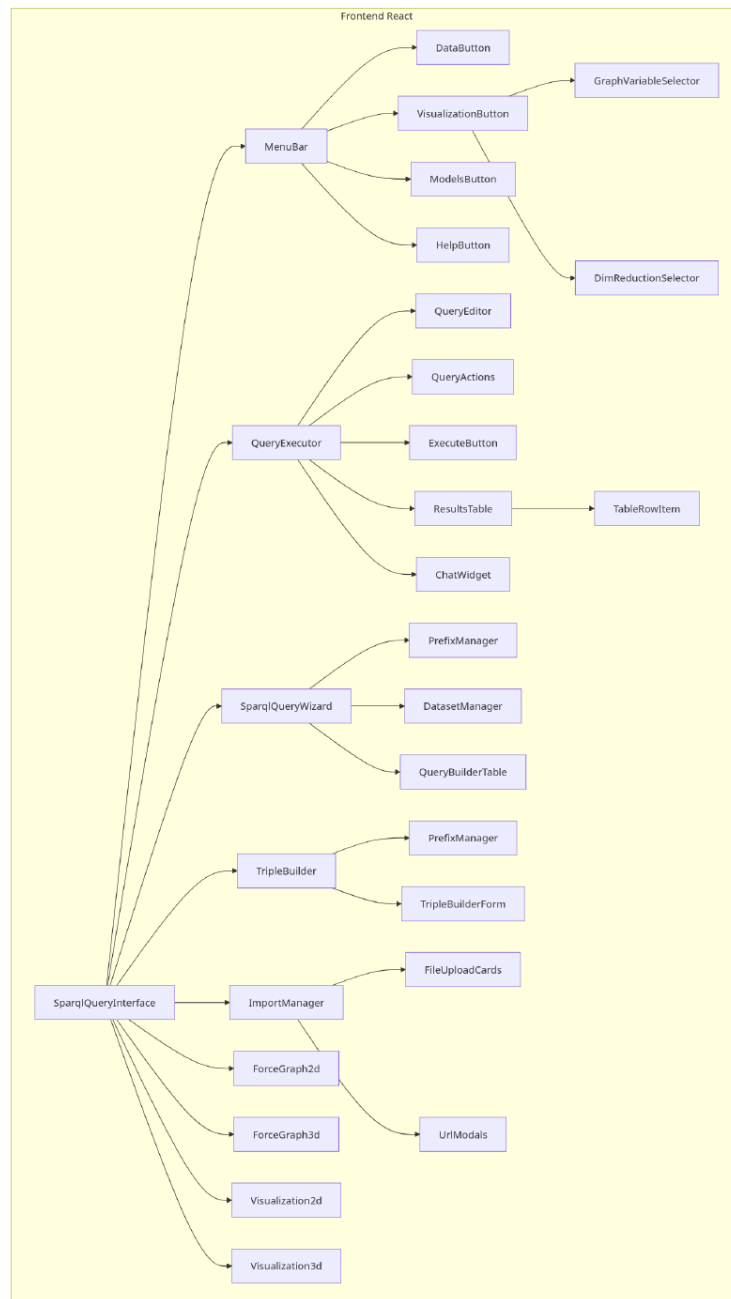


Figura 37: Jerarquía de componentes del frontend

28.1. Arquitectura General y Enrutamiento

El componente raíz es `SparqlQueryInterface.tsx`, que gestiona el enrutamiento (usando React Router) y el estado global de la aplicación (consulta actual, resultados, mensajes).

28.1.1. Vistas Principales (Rutas)

- `/executor`: Ejecutor de consultas SPARQL.
- `/wizard`: Asistente visual para construir consultas.
- `/builder`: Constructor de triplas RDF.
- `/graph2d`, `/graph3d`: Visualización de grafos.
- `/visuals2d`, `/visuals3d`: Visualización de reducción dimensional.
- `/importer`: Gestor de importación de datos.

28.2. Componentes de Visualización

28.2.1. Visualización de Grafos

graph-3d-view.tsx: Utiliza Three.js y la biblioteca **3d-force-graph** para renderizar grafos interactivos en WebGL. Soporta interacción (zoom, rotación, clic en nodos) y estilizado (tamaños, colores, etiquetas).

graph-2d-view.tsx: Renderiza grafos en 2D usando un iframe que carga la biblioteca **force-graph**. Este aislamiento previene conflictos de CSS/JS.

28.2.2. Visualización de Reducción Dimensional

reduction-view-2d.tsx: Muestra *embeddings* 2D usando D3.js y Canvas HTML5. Incluye zoom, panorámica, selección rectangular (*brush*) y tooltips.

reduction-view-3d.tsx: Muestra *embeddings* 3D usando Three.js. Implementa un sistema completo de iluminación, controles de órbita y tooltips interactivos. Los colores se mapean desde un gradiente basado en las coordenadas normalizadas.

28.3. Componentes de Gestión de Datos y Consultas

28.3.1. Importación de Datos

import-manager.tsx: Interfaz para subir archivos (arrastrar y soltar) o conectar a URLs/BDs. Se comunica con el backend Python Importer (puerto 5001) y permite generar y descargar el RDF resultante.

28.3.2. Ejecución de Consultas

query-executor.tsx: Componente principal que integra un editor de consultas SPARQL, una tabla de resultados paginada y un widget de chat con Botpress (asistente de IA). Envía consultas al backend Java (puerto 8080) y parsea la respuesta CSV usando PapaParse.

28.3.3. Construcción de Consultas

sparql-query-wizard.tsx: Asistente visual tipo tabla que permite construir consultas SPARQL complejas (SELECT, filtros, agregaciones, patrones OPTIONAL/UNION) sin escribir código.

triple-builder.tsx: Constructor visual de triplas RDF para generar sentencias **INSERT DATA**. Maneja prefijos y tipado automático de literales.

28.4. Componentes de Interfaz y Navegación

28.4.1. Barra de Navegación y Botones de Acción

- **menu-bar.tsx:** Barra superior con logo y botones principales.
- **visualization-button.tsx:** Menú desplegable que centraliza el acceso a todas las opciones de visualización (grafos, reducción dimensional). Activa selectores de variables para configurar los gráficos.
- **data-button.tsx:** Gestiona operaciones con el dataset (abrir desde múltiples fuentes, exportar a 9 formatos RDF, limpiar).
- **models-button.tsx:** Gestiona la creación de modelos ontológicos y de inferencia en el backend Java.
- **execute-button.tsx:** Botón especializado con estados (normal, cargando) para ejecutar consultas.

29. Integración y Flujos de Trabajo

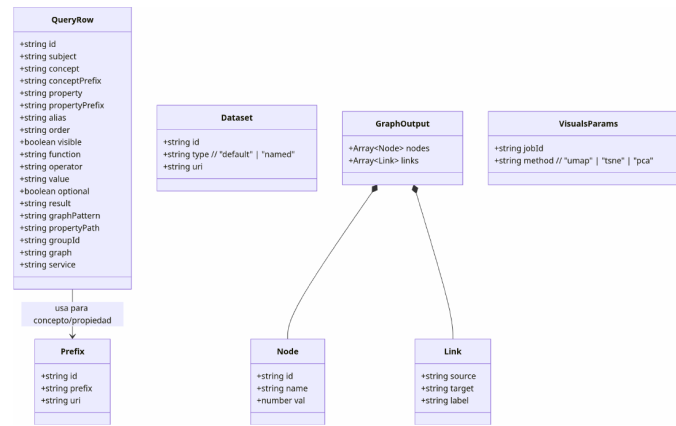


Figura 38: Diagrama de entidades principales del frontend

29.1. Comunicación entre Componentes

- *Frontend ↔ Backend Java (Puerto 8080)*: Para todas las operaciones SPARQL, gestión del grafo RDF, y creación de modelos de razonamiento.
- *Frontend ↔ Backend Python - Importer (Puerto 5001)*: Para subir archivos estructurados y obtener su representación RDF.
- *Frontend ↔ Backend Python - Visualizer (Puerto 5000)*: Para enviar datos y obtener *embeddings* 2D/3D.

Los estados globales en React (consulta, resultados) son compartidos entre los componentes relacionados para una experiencia coherente.

29.2. Flujo de Trabajo Típico

1. *Importar Datos*: El usuario sube un archivo CSV mediante el **import-manager** (Python Importer) o carga un grafo RDF existente mediante el **data-button** (Java).
2. *Consultar/Explorar*: Construye una consulta en el **query-wizard** o **executor** y la ejecuta. Los resultados tabulares quedan disponibles.
3. *Visualizar*: Desde el **visualization-button**, elige:
 - *Gráfico del Grafo*: Selecciona variables para nodos/enlaces y visualiza en 2D/3D.
 - *Reducción Dimensional*: Selecciona columnas y algoritmo. Los datos se envían al Visualizer de Python y los resultados se muestran en las vistas 2D/3D.
4. *Exportar/Guardar*: Usa el **data-button** para exportar el grafo de conocimiento o los resultados en diversos formatos.

30. Docker: Contenerización y Orquestación de Servicios

El proyecto utiliza Docker y Docker Compose para empaquetar, distribuir y orquestar todos los componentes del sistema. Este enfoque garantiza consistencia entre entornos de desarrollo, prueba y producción, simplifica la instalación de dependencias y facilita el despliegue escalable.

30.1. Arquitectura de Contenedores

El sistema se estructura en cuatro servicios independientes definidos en el archivo **docker-compose.yaml**:

- *importer-service*: Contiene el microservicio Python de importación de datos (puerto 5001).
- *visualizer-service*: Contiene el microservicio Python de reducción dimensional (puerto 5000).

- *sparql-backend-service*: Contiene la aplicación Java Spring Boot con Apache Jena (puerto 8080).
- *frontend-service*: Contiene la aplicación React/Vite servida mediante **http-server** (puerto 80).

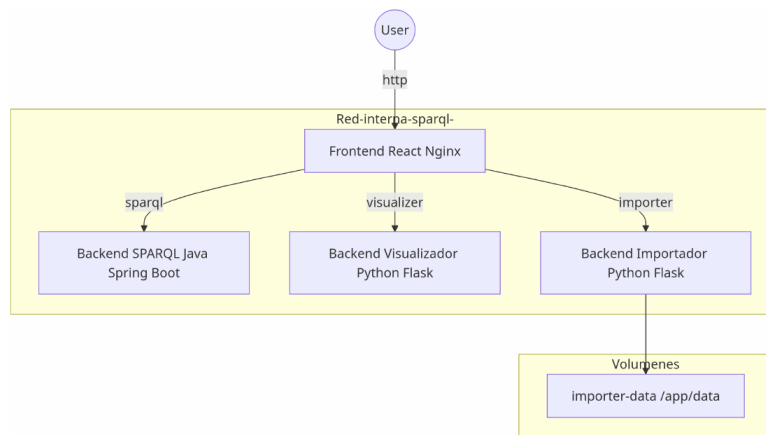


Figura 39: Diagrama de despliegue de la aplicación con docker compose

30.2. Configuración de Redes

Los servicios se comunican a través de una red puente personalizada llamada **sparql-network**:

```

networks:
  sparql-network:
    driver: bridge
  
```

Esta configuración permite:

- Aislamiento de la red del sistema host.
- Comunicación interna entre contenedores usando los nombres de servicio como nombres de host (DNS automático).
- Exposición controlada de puertos al host solo donde es necesario.

30.3. Políticas de Reinicio

Todos los servicios están configurados con **restart: unless-stopped**, lo que garantiza:

- Reinicio automático si el contenedor falla (excepto si fue detenido manualmente).
- Mayor disponibilidad del sistema sin intervención manual.
- Recuperación ante fallos del host o del demonio Docker.

30.4. Flujo de Construcción y Despliegue

Cada servicio tiene su propio **Dockerfile** en su directorio correspondiente, permitiendo construcciones independientes. El flujo típico de despliegue es:

1. *Construcción de imágenes:*

```
docker-compose build
```

2. *Inicio de servicios:*

```
docker-compose up -d
```

3. *Monitoreo:*

```
docker-compose logs -f
```

4. Detención:

```
docker-compose down
```

30.5. Ventajas de la Contenerización en este Proyecto

La arquitectura basada en contenedores ofrece beneficios específicos para este sistema:

- *Aislamiento de Dependencias*: Cada servicio tiene sus propias dependencias (Java JDK, Python 3.9, Node.js) sin conflictos.
- *Escalabilidad Independiente*: Los servicios pueden escalarse por separado según la carga (ej: múltiples instancias del visualizer).
- *Portabilidad*: El sistema completo puede ejecutarse en cualquier host con Docker, sin necesidad de instalar manualmente Java, Python o Node.js.
- *Versionado Consistente*: Las versiones exactas de cada dependencia se fijan en los Dockerfiles, garantizando reproducibilidad.
- *Integración con CI/CD*: La contenerización facilita la implementación de pipelines de integración y despliegue continuo.

30.6. Consideraciones de Configuración

- *Persistencia de Datos*: Los contenedores son efímeros por defecto. Para producción, se deberían añadir volúmenes Docker para datos persistentes.
- *Seguridad*: Los contenedores se ejecutan con usuarios no privilegiados cuando es posible, y solo los puertos necesarios están expuestos.
- *Rendimiento*: La red bridge ofrece buen rendimiento para comunicación inter-servicios, con latencia mínima.
- *Recursos*: Es posible limitar CPU y memoria por contenedor mediante configuraciones adicionales en el docker-compose.

30.7. Ejemplo de Comunicación entre Contenedores

Dentro del ecosistema Docker, los servicios se comunican usando los nombres definidos:

```
// Desde el frontend, llamar al backend Java
fetch('http://sparql-backend-service:8080/sparql/runQuery', ...)

// Desde el backend Java, llamar al importador Python
// (si fuera necesario)
// http://importer-service:5001/dataframes
```

Esta configuración permite que el frontend, ejecutándose en un contenedor separado, se comunique con el backend Java utilizando simplemente el nombre del servicio (`sparql-backend-service`) en lugar de localhost o una IP específica.

30.8. Configuración Individual de Contenedores

Cada servicio cuenta con un `Dockerfile` específico optimizado para sus necesidades, asegurando un entorno de ejecución eficiente y seguro. A continuación, se detalla la configuración de cada uno:

30.8.1. Frontend (React/Vite)

El contenedor del frontend se construye a partir de una imagen ligera de Node.js Alpine y sigue un proceso de construcción en dos etapas:

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --only=production
RUN npm install -g typescript @types/node vite
COPY . .
RUN npm run build
RUN npm install -g http-server
EXPOSE 80
CMD ["http-server", "dist", "-p", "80"]
```

Características clave:

- *Tamaño reducido:* Utiliza la variante **alpine** de Node.js para minimizar la imagen final.
- *Construcción separada:* Instala las dependencias de producción, luego instala globalmente TypeScript y Vite necesarios para la construcción (**build**), y finalmente compila la aplicación React.
- *Servidor estático:* Utiliza **http-server** para servir los archivos estáticos generados en la carpeta **dist** en el puerto 80.
- *Optimización:* La instalación de dependencias se realiza antes de copiar el código fuente para aprovechar la caché de capas de Docker.

30.8.2. Backend Java (Spring Boot con Apache Jena)

El backend Java utiliza una construcción multi-etapa para separar la fase de compilación de la de ejecución, reduciendo el tamaño final de la imagen:

```
----- Build stage -----

FROM maven:3.9.6-eclipse-temurin-21 AS build
WORKDIR /build
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests
----- Runtime stage -----

FROM eclipse-temurin:21-jdk-jammy
WORKDIR /app
COPY --from=build /build/target/backend-1.0.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Características clave:

- *Construcción multi-etapa:* La primera etapa (**build**) usa Maven para compilar el proyecto y generar el archivo JAR. La segunda etapa (**runtime**) utiliza solo el JDK para ejecutar la aplicación.
- *Optimización de dependencias:* El comando **mvn dependency:go-offline** descarga y cachea las dependencias, acelerando construcciones posteriores.
- *Imagen de ejecución ligera:* La etapa de runtime solo incluye el JAR y el JDK, sin las herramientas de construcción (Maven).
- *Java 21:* Utiliza la distribución Eclipse Temurin de Java 21, asegurando compatibilidad con las últimas características del lenguaje y rendimiento.

30.8.3. Microservicios Python (Visualizer e Importer)

Ambos servicios Python comparten una configuración similar, optimizada para desempeño y seguridad:

```
FROM python:3.13.5-slim
WORKDIR /app
RUN apt-get update &&
apt-get install -y --no-install-recommends
gcc g++ libopenblas-dev libatlas3-base
&& rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY visualizer.py . # o importer.py
EXPOSE 5000 # o 5001
ENV PYTHONUNBUFFERED=1
PYTHONDONTWRITEBYTECODE=1
RUN adduser --disabled-password --gecos '' appuser
USER appuser
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3
CMD ["unicorn", "--bind", "0.0.0.0:5000", "visualizer:app",
"--workers", "1", "--threads", "4", "--log-level", "info"]
```

Características clave:

- *Imagen base ligera:* Se utiliza la variante `slim` de Python 3.13.5 y `alpine`.
- *Dependencias del sistema:* Se instalan compiladores y bibliotecas matemáticas (`libopenblas-dev`) necesarias para el rendimiento óptimo de NumPy, SciPy y scikit-learn utilizados en los algoritmos de reducción dimensional.
- *Gestión de dependencias Python:* Se copia el archivo `requirements.txt` y se instalan las dependencias con `pip`, utilizando la bandera `--no-cache-dir` para reducir el tamaño de la imagen.
- *Seguridad:*
 - Se establecen variables de entorno para desactivar el buffering de salida (`PYTHONUNBUFFERED`) y la escritura de bytecode (`PYTHONDONTWRITEBYTECODE`).
 - Se crea y utiliza un usuario no privilegiado (`appuser`) para la ejecución de la aplicación, minimizando riesgos en caso de vulnerabilidades.
- *Healthcheck integrado:* Cada contenedor incluye una verificación de salud a nivel de Docker que prueba la disponibilidad del endpoint raíz del servicio.
- *Servidor de producción:* Se utiliza `unicorn` como servidor WSGI con configuración predefinida (1 worker, 4 threads) para manejar múltiples peticiones concurrentes de manera eficiente.

30.9. Integración de los Dockerfiles en el Flujo de Construcción

El archivo `docker-compose.yaml` referencia estos `Dockerfile` para construir (o descargar) las imágenes necesarias. Durante el desarrollo, se puede utilizar `docker-compose up --build` para reconstruir y relanzar todos los servicios. Esta configuración asegura que:

- *Dependencias Aisladas:* Cada servicio tiene su propio entorno con versiones específicas de Node.js, Java o Python, evitando conflictos.
- *Construcciones Reproducibles:* Los `Dockerfile` garantizan que cada construcción produzca una imagen idéntica, independientemente del entorno del desarrollador.
- *Despliegue Eficiente:* Las imágenes resultantes están optimizadas en tamaño (especialmente la etapa final del backend Java y el uso de variantes `alpine/slim`) y listas para producción.

30.10. Consideraciones de Seguridad y Buenas Prácticas Implementadas

La configuración de los Dockerfiles sigue varias buenas prácticas de seguridad y rendimiento:

- *Usuarios no root*: Los servicios Python se ejecutan como un usuario sin privilegios (**appuser**). El frontend, al usar **http-server** en Alpine, también se ejecuta de manera no privilegiada por defecto. El backend Java hereda la configuración de usuario de la imagen base de Eclipse Temurin.
- *Healthchecks*: Implementados directamente en los Dockerfiles de Python y referenciados en el compose, permiten a Docker y a orquestadores supervisar el estado del servicio.

31. Uso de Nginx como proxy inverso

Para facilitar la comunicación entre el frontend y los distintos microservicios backend, se ha implementado un servidor **Nginx** que actúa como proxy inverso. Esta decisión arquitectónica permite centralizar el enrutamiento de las peticiones HTTP, mejorar la seguridad y simplificar la configuración del cliente.

31.1. Arquitectura de red y servicios

La aplicación se despliega mediante **Docker Compose**, creando una red interna denominada **sparql-network** en la que conviven los siguientes contenedores:

- **frontend-service**: sirve los archivos estáticos de la aplicación React.
- **sparql-backend-service**: backend en Java Spring Boot que gestiona consultas SPARQL y manipulación de datasets.
- **importer-service**: backend en Python Flask encargado de importar datos estructurados y convertirlos a RDF.
- **visualizer-service**: backend en Python Flask que calcula embeddings mediante técnicas de reducción de dimensionalidad (PCA, t-SNE, UMAP).

El contenedor de Nginx forma parte del servicio **frontend** y expone el puerto 80 hacia el exterior, actuando como puerta de entrada única para todas las peticiones.

31.2. Configuración del proxy inverso

El archivo **nginx.conf** define las reglas de enrutamiento. A continuación se muestra un fragmento relevante:

```
location /api/sparql/ {
    proxy_pass http://sparql-backend:8080/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Cada bloque **location** captura las rutas que comienzan con un prefijo específico (**/api/sparql/**, **/api/importer/**, **/api/visualizer/**) y redirige la petición al contenedor correspondiente utilizando su nombre de servicio interno y el puerto expuesto. Las directivas **proxy_set_header** garantizan que la información original del cliente (dirección IP, protocolo) se transmita a los backend, lo que resulta útil para registros y depuración.

Además del proxy, Nginx también sirve los archivos estáticos de la aplicación React. La directiva **try_files \$uri \$uri/ /index.html;** asegura que cualquier ruta no coincidente con los prefijos de API sea manejada por el enrutamiento del lado del cliente (SPA).

31.3. Ventajas de este enfoque

- **Punto de entrada único:** El frontend solo necesita conocer la dirección del servidor Nginx; no requiere conocer las direcciones individuales de cada backend. Esto simplifica la configuración en el entorno de desarrollo y producción.
- **Separación de dominios:** Todas las peticiones a los backend se realizan bajo el mismo dominio y puerto, evitando problemas de CORS (Cross-Origin Resource Sharing) que surgirían si el frontend intentara acceder directamente a servicios en puertos diferentes.
- **Seguridad:** Los backend no están expuestos directamente a Internet; solo son accesibles a través de la red interna de Docker. Nginx actúa como un firewall de aplicaciones, pudiendo añadir capas adicionales de autenticación, limitación de tasa o registro de accesos.
- **Flexibilidad:** En el futuro, se podrían añadir más servicios backend sin modificar el frontend, simplemente agregando nuevas reglas de proxy en Nginx.

31.4. Integración con Docker Compose

En el archivo `docker-compose.yaml`, el servicio `frontend` se construye con una imagen que incluye Nginx y los archivos estáticos. Las variables de entorno `VITE_*` definen las rutas relativas que el frontend utilizará para construir las URLs de las API (por ejemplo, `/api/sparql/runQuery`). Gracias al proxy inverso, estas rutas son transparentemente redirigidas al backend correspondiente.

```
frontend:
  build:
    context: ./Frontend
  args:
    VITE_SPARQL_BACKEND_URL: /api/sparql
    VITE_IMPORTER_URL: /api/importer
    VITE_VISUALIZER_URL: /api/visualizer
  ports:
    - "80:80"
  networks:
    - sparql-network
```

De esta manera, la aplicación en el navegador realiza peticiones a rutas como `/api/sparql/runQuery`, que Nginx interpreta y redirige internamente a `http://sparql-backend:8080/runQuery`. Este diseño mantiene la coherencia y facilita el despliegue en entornos donde el puerto 80 es el estándar para el tráfico web.

Bibliografía

- [1] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia D'Amato, Gerard De Melo, Claudio Gutierrez, José Emilio Labra Gayo, Sabrina Kirrane, Sebastian Neumaier, Axel Polleres, Roberto Navigli, Axel-Cyrille Ngonga Ngomo, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan F. Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs, 2020. arXiv preprint. [arXiv:2003.02320](https://arxiv.org/abs/2003.02320).
- [2] Bob Ducharme. *Learning SPARQL*. O'Reilly Media, 2 edition, 2013.
- [3] Wikipedia contributors. Linked data, 2025. Accessed: 2026-02-09. URL: https://en.wikipedia.org/wiki/Linked_data.
- [4] Wikipedia contributors. Knowledge graph, 2025. Accessed: 2026-02-09. URL: https://en.wikipedia.org/wiki/Knowledge_graph.
- [5] Wikipedia contributors. Knowledge graph (google), 2025. Accessed: 2026-02-09. URL: [https://en.wikipedia.org/wiki/Knowledge_Graph_\(Google\)](https://en.wikipedia.org/wiki/Knowledge_Graph_(Google)).
- [6] Wikipedia contributors. Resource description framework, 2025. Accessed: 2026-02-09. URL: https://en.wikipedia.org/wiki/Resource_Description_Framework.
- [7] Apache Software Foundation. Apache jena. Accessed: 2026-02-09. URL: <https://jena.apache.org/>.

- [8] Apache Software Foundation. Apache maven. Accessed: 2026-02-09. URL: <https://maven.apache.org/>.
- [9] VMware, Inc. Spring boot. Accessed: 2026-02-09. URL: <https://spring.io/projects/spring-boot>.
- [10] Evan You and contributors. Vite. Accessed: 2026-02-09. URL: <https://vite.dev/>.
- [11] Inc. Docker. Docker. Accessed: 2026-02-09. URL: <https://www.docker.com/>.
- [12] Meta Open Source. React. Accessed: 2026-02-09. URL: <https://react.dev/>.
- [13] Microsoft. Typescript. Accessed: 2026-02-09. URL: <https://www.typescriptlang.org/>.
- [14] Three.js Authors. Three.js. Accessed: 2026-02-09. URL: <https://threejs.org/>.
- [15] Michael Bostock, Vadim Ogievetsky, and Jeffrey Heer. D3.js - data-driven documents. Accessed: 2026-02-09. URL: <https://d3js.org/>.
- [16] Vasco Vasconcelos. Force graph. Accessed: 2026-02-09. URL: <https://github.com/vasturiano/force-graph>.
- [17] Botpress. Botpress - conversational ai platform. Accessed: 2026-02-09. URL: <https://botpress.com/>.
- [18] Pallets Projects. Flask. Accessed: 2026-02-09. URL: <https://flask.palletsprojects.com/>.
- [19] Unicorn Authors. Unicorn - python wsgi http server. Accessed: 2026-02-09. URL: <https://unicorn.org/>.
- [20] scikit-learn Developers. scikit-learn. Accessed: 2026-02-09. URL: <https://scikit-learn.org/stable/>.
- [21] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection. Accessed: 2026-02-09. URL: <https://umap-learn.readthedocs.io/>.
- [22] F5, Inc. nginx, 2025. Accessed: 2026-02-09. URL: <https://nginx.org/>.